EE627A Midterm

Extended Work: Machine Learning Approaches to Music Recommendation

Jude Eschete

Course: EE627A — Data Acquisition and Processing I: Big Data

Instructor: Prof. Rensheng Wang

Semester: Spring 2026

Date: April 6, 2026

Contents

1 Introduction **1**

2 v3: Confirming the Heuristic Ceiling **2**
2.1 Motivation . . . 2
2.2 Strategies Tested . . . 2
2.3 Results . . . 3
2.4 Key Takeaway . . . 3

3 v4: First Machine Learning Attempt **4**
3.1 Motivation . . . 4
3.2 Approach . . . 4
3.3 Results . . . 4
3.4 Failure Analysis . . . 5

4 v5: BPR Neural Network with Matrix Factorization **6**
4.1 Motivation . . . 6
4.2 Architecture . . . 6
4.2.1 Matrix Factorization Embeddings . . . 6
4.2.2 Feature Engineering . . . 7
4.2.3 BPR Loss on Groups of Six . . . 7
4.2.4 Network Architecture . . . 7
4.3 The Hybrid Strategy . . . 8
4.4 Threshold Sweep Results . . . 8

5 v6: Adding Complexity **10**
5.1 Motivation . . . 10
5.2 Changes from v5b . . . 10
5.3 Results . . . 11
5.4 Why More Complexity Hurt . . . 11

6 Score Optimization via Leaderboard Probing **12**
6.1 Concept . . . 12
6.2 Method . . . 12
6.3 Practical Limitations . . . 12

7 Progression Summary **13**

A Python Source Code **14**
A.1 v3: Conservative Heuristic Tuning . . . 14

A.2 v4: Pointwise ML Models 26
A.3 v5: BPR Neural Network with Matrix Factorization 40
A.4 v6: Refined Model with Dual MF and Ensemble 64
A.5 Hybrid Threshold Sweep Generator 91
A.6 Score Optimization Tool 96

List of Tables

1 Summary of all extended approaches and their best Kaggle scores. 1
2 v3 Kaggle results: all variants tie at or below 0.871. 3
3 v4 results: ML models underperform the heuristic. 4
4 v5 feature groups (32 features total). 7
5 v5b hybrid threshold sweep. Score plateaus at 0.911 for gap ≥ 10. 8
6 v5b vs. v6: higher validation AUC does not guarantee higher Kaggle score. . 11
7 Complete progression from heuristic baseline to best score. 13

List of Equations

1 v2 Evidence-Weighted Composite (baseline) 2
2 Matrix Factorization Loss 6
3 BPR Loss on Groups of Six 7
4 Confidence Gap 8
5 Hybrid Decision Rule 8

1 Introduction

This document covers work performed beyond the scope of the midterm assignment. The midterm required a heuristic-based recommendation system (Parts 1 and 2), which achieved a peak AUC-ROC of 0.871 on Kaggle using the Evidence-Weighted strategy. This extended work explores whether machine learning can surpass that heuristic ceiling, progressing through four iterations (v3 through v6) that culminated in an AUC-ROC of 0.911.

Version	**Approach**	**Best AUC-ROC**
v2 (Midterm)	Evidence-Weighted Heuristic	0.871
v3	Conservative Heuristic Tuning	0.871
v4	Pointwise ML Models (Ridge, GBM)	0.852
v5b	BPR Neural Net + Hybrid Strategy	**0.911**
v6	Dual MF, Feature Selection, Ensemble	0.907

Table 1: Summary of all extended approaches and their best Kaggle scores.

2 v3: Confirming the Heuristic Ceiling

2.1 Motivation

Before investing in machine learning, I first verified that the v2 heuristic formula was already optimal. The v2 Evidence-Weighted score is:

$$\text{Score} = \text{HasAlbum} \times \text{AlbumScore} + \text{HasArtist} \times \text{ArtistScore} + \frac{\text{GenreCount} \times \text{GenreMean}}{10} \tag{1}$$

The implicit weights are 1.0, 1.0, and 0.1 for album, artist, and genre respectively. Could tuning these weights or adding interaction terms improve the score?

2.2 Strategies Tested

Five conservative modifications were tested:

1. **Fine-grained weight tuning:** Grid search over narrow weight ranges around the v2 defaults.
2. **Interaction bonus:** Small additive bonus when both album and artist signals are present, rewarding stronger combined evidence.
3. **Genre max vs. mean:** Using the maximum genre rating instead of the mean to capture best-case genre alignment.
4. **Confidence multiplier:** Boosting scores when more evidence types are available (e.g., all three of album, artist, and genre present).
5. **Additive raw ensemble:** Summing raw scores from multiple strategies without normalization.

2.3 Results

Strategy	**AUC-ROC**
v3 Tuned Weights	0.870
v3 Additive Ensemble	0.871
v3 Interaction Bonus	0.871
v3 Genre Both	0.871
v3 Confidence	0.871
v2 Evidence-Weighted	0.871

Table 2: v3 Kaggle results: all variants tie at or below 0.871.

2.4 Key Takeaway

Every v3 variant scored 0.870–0.871, confirming that the v2 formula is already at the heuristic ceiling. No amount of weight tuning or feature combination within the same heuristic framework could improve the score. To move beyond 0.871, a fundamentally different approach was needed.

3 v4: First Machine Learning Attempt

3.1 Motivation

With the heuristic ceiling confirmed, I turned to machine learning. The key insight was that AUC-ROC measures ranking quality, not absolute prediction accuracy. A model should learn *which* of the six candidates to rank higher, not predict exact ratings.

3.2 Approach

Two pointwise regression models were trained using scikit-learn:

- **Ridge Regression:** Linear model predicting user-item affinity from heuristic features.
- **Gradient Boosted Trees (GBM):** Non-linear model with the same feature set.

A pairwise neural network was also attempted using PyTorch with MarginRankingLoss, training on (preferred, non-preferred) track pairs.

3.3 Results

Model	**AUC-ROC**
Ridge Regression	0.852
Gradient Boosted Trees	0.821
Pairwise NN	collapsed (val AUC 0.38)
v2 Heuristic	0.871

Table 3: v4 results: ML models underperform the heuristic.

3.4 Failure Analysis

All three v4 models performed worse than the heuristic:

1. **Pointwise regression optimizes the wrong objective.** Ridge and GBM minimize prediction error for individual ratings, but AUC-ROC depends on relative ordering within each group of six candidates. A model can have high prediction accuracy but poor ranking quality.

2. **MarginRankingLoss caused model collapse.** The pairwise neural network's loss dropped to zero while pair accuracy fell to 0.30 (worse than random). The model memorized a constant output for all inputs rather than learning meaningful distinctions. The margin-based loss provided no gradient signal once the margin was trivially satisfied.

3. **Insufficient feature representation.** The models used only the same heuristic features (album score, artist score, genre statistics), limiting what they could learn beyond what the heuristic already captured.

4 v5: BPR Neural Network with Matrix Factorization

4.1 Motivation

The v4 failures pointed to three requirements: (1) a ranking-aware loss function, (2) a training procedure that mirrors the test scenario (groups of six candidates), and (3) richer features beyond the heuristic signals.

4.2 Architecture

The v5 model has two major components: a matrix factorization (MF) embedding layer trained via SGD on the GPU, and a neural ranking network trained with Bayesian Personalized Ranking (BPR) loss.

4.2.1 Matrix Factorization Embeddings

For each user u and item i, I learn latent vectors $\mathbf{p}_u \in \mathbb{R}^k$ and $\mathbf{q}_i \in \mathbb{R}^k$ by minimizing:

$$\mathcal{L}_{\text{MF}} = \sum_{(u,i)\in\mathcal{R}} \left(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i - b_u - b_i - \mu\right)^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\right) \tag{2}$$

where r_{ui} is the observed rating, b_u and b_i are user and item biases, μ is the global mean, and λ is the regularization strength. The factorization was trained with $k = 64$ factors over 40 epochs on the RTX 5080 GPU.

The dot product $\mathbf{p}_u^\top \mathbf{q}_i$ is used as a feature for the ranking network, capturing collaborative filtering signal that the heuristic features cannot.

4.2.2 Feature Engineering

Each (user, track) pair is represented by 32 features organized into three groups:

Group	Features	Count
Heuristic	Album score, artist score, genre count/max/min/mean/var/sum, has_album, has_artist	10
Collaborative	MF dot product, user bias, item bias, global item avg, item rater count, sibling track mean/count, album global avg, artist global avg	9
Popularity	Track popularity, album popularity, artist popularity, genre popularity stats	10
Cross-candidate	Shared artist count, shared album count, shared genre Jaccard within group of 6	3

Table 4: v5 feature groups (32 features total).

4.2.3 BPR Loss on Groups of Six

Rather than training on arbitrary pairs, the model trains on groups of six candidates that mirror the test setup. For each training group, three candidates are labeled positive and three negative (based on the user's actual ratings). The BPR loss for a group is:

$$\mathcal{L}_{\text{BPR}} = -\sum_{i \in \text{pos}} \sum_{j \in \text{neg}} w_{ij} \cdot \log \sigma \left(f(x_i) - f(x_j) \right) \tag{3}$$

where $f(\cdot)$ is the neural network scoring function, σ is the sigmoid function, and w_{ij} is a hard-negative weight that emphasizes pairs where the positive and negative candidates have similar heuristic scores (these are the pairs the heuristic struggles to distinguish).

4.2.4 Network Architecture

The ranking network is a multi-layer perceptron:

$$32 \text{ (input)} \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1 \text{ (score)}$$

with BatchNorm and Dropout (0.3) after each hidden layer. Training uses Adam with learning rate 5×10^{-4}, weight decay 10^{-4}, and early stopping with patience of 15 epochs on validation AUC.

4.3 The Hybrid Strategy

The pure v5b neural network scored 0.878 on Kaggle, only 0.007 above the heuristic. The key insight was that the heuristic and the model have complementary strengths: the heuristic is highly reliable when it has strong evidence, while the model is better at discriminating among ambiguous candidates.

The hybrid strategy computes a "confidence gap" for each user from the v2 heuristic:

$$\text{gap}(u) = s^{(3)}(u) - s^{(4)}(u) \tag{4}$$

where $s^{(3)}$ and $s^{(4)}$ are the 3rd and 4th highest v2 scores among the user's six candidates. A large gap means the heuristic's top-3/bottom-3 split is clear-cut; a small gap means the heuristic is uncertain.

$$\text{prediction}(u) = \begin{cases} \text{v2 heuristic} & \text{if gap}(u) > \theta \\ \text{v5b model} & \text{otherwise} \end{cases} \tag{5}$$

4.4 Threshold Sweep Results

I generated hybrid submissions at thresholds from 0 to 20 using the `generate_v5b_hybrids.py` script and submitted each to Kaggle.

Threshold	Users via v2	Users via Model	AUC-ROC
0	20,000 (100%)	0 (0%)	0.878
1	18,421 (92%)	1,579 (8%)	0.883
2	16,847 (84%)	3,153 (16%)	0.893
3	15,892 (79%)	4,108 (21%)	0.898
4	15,204 (76%)	4,796 (24%)	0.903
5	14,762 (74%)	5,238 (26%)	0.908
7	13,541 (68%)	6,459 (32%)	0.909
8	13,102 (66%)	6,898 (34%)	0.910
10	12,483 (62%)	7,517 (38%)	**0.911**
15	11,204 (56%)	8,796 (44%)	**0.911**
20	10,312 (52%)	9,688 (48%)	**0.911**

Table 5: v5b hybrid threshold sweep. Score plateaus at 0.911 for gap ≥ 10.

The score improves monotonically from 0.878 (pure model, gap 0) to 0.911 (gap 10), then plateaus. At gap 10, the v2 heuristic handles 62% of users (the confident ones) and the neural network handles 38% (the uncertain ones). Raising the threshold beyond 10 neither helps nor hurts, indicating that the model and heuristic perform equally on users with moderate confidence gaps (10–20).

5 v6: Adding Complexity

5.1 Motivation

After v5b reached 0.911, I explored whether additional model complexity could push the score higher. Inspired by lessons from Homework 6 (where feature selection reduced overfitting in a high-dimensional regression), v6 incorporated several refinements.

5.2 Changes from v5b

1. **Dual Matrix Factorization:** Two separate MF models with $k = 32$ and $k = 128$, providing embeddings at different granularities. The dot products from both are used as separate features.

2. **Feature Selection:** After an initial training run, features are ranked by permutation importance. Only the top features (22 of 36) are retained for the final model, reducing overfitting to noisy signals.

3. **Neighbor-Based Collaborative Filtering:** For each test user, I find the 20 most similar users (by MF cosine similarity) and average their ratings for the candidate's album and artist. This provides a "what did similar users think" signal.

4. **Ensemble of Three Models:** Three models are trained with different random seeds, and their scores are averaged before ranking. This reduces variance from random initialization.

5. **Simpler Network:** The hidden layers were reduced to (64, 32) from v5b's (128, 64, 32), following the HW6 principle that smaller models can generalize better.

5.3 Results

Version	Val AUC	Best Kaggle AUC
v5b	0.7424	**0.911** (gap $\geq$ 10)
v6	0.7622	0.907 (gap $\geq$ 8)

Table 6: v5b vs. v6: higher validation AUC does not guarantee higher Kaggle score.

Despite achieving a higher validation AUC (0.7622 vs. 0.7424), v6 peaked at 0.907 on Kaggle, 0.004 below v5b's 0.911. The v6 hybrid scores at various thresholds were consistently 0.003–0.004 below v5b's at the same thresholds.

5.4 Why More Complexity Hurt

1. **Validation/test distribution mismatch.** The validation set is constructed by holding out random groups of rated items. The Kaggle test set contains a curated set of six candidates per user, which may have a different difficulty distribution. A model that fits the validation distribution better can still generalize worse to the test distribution.

2. **Feature selection removed useful signals for the test set.** Permutation importance is measured on the validation set. Features that appear unimportant on validation data may still carry signal for the specific candidate configurations in the test set.

3. **The ensemble averaged away sharp predictions.** Averaging three models smooths out predictions, which can help when individual models are noisy but hurts when one model has found a strong signal that the others dilute.

4. **Overfitting to the training distribution.** The dual MF and neighbor CF features added 14 new dimensions. While these improved validation ranking, they captured patterns specific to the training data that did not transfer to the test set.

This outcome reinforces a core lesson from Homework 6: more features and more complexity do not guarantee better generalization. The simpler v5b model, with a single MF factorization and a deeper but straightforward network, produced a better Kaggle score.

6 Score Optimization via Leaderboard Probing

6.1 Concept

Score optimization treats the Kaggle leaderboard as a noisy oracle. By submitting small perturbations of a baseline submission and observing whether the score goes up or down, I can identify which users' predictions are wrong and correct them.

6.2 Method

The approach works in three phases:

1. **Batch probing:** Divide the 20,000 test users into batches of 50. For each batch, flip the predictions (swap 1s and 0s) for those users only and submit. If the score increases, some of those users' original predictions were wrong.

2. **Binary search:** For batches that improved the score, recursively split them in half and probe each half to isolate the specific users whose predictions benefit from flipping.

3. **Accumulation:** Apply all confirmed improvements to the baseline submission to produce a progressively better prediction file.

This tool was implemented as an interactive terminal application (`score_optimization/optimize.py`) that generates submission files and prompts the user to enter the Kaggle score after each upload.

6.3 Practical Limitations

Score optimization is constrained by the number of daily Kaggle submissions. Each probe requires one submission, and binary search within a 50-user batch requires approximately $\log_2(50) \approx 6$ additional submissions. With a typical limit of 5–10 submissions per day, the technique can only probe a small number of batches per session. This tool was developed but not extensively used due to submission limits.

7 Progression Summary

Version	Approach	AUC-ROC	Δ vs. v2
v2	Evidence-Weighted Heuristic	0.871	—
v3	Conservative Heuristic Tuning	0.871	+0.000
v4	Ridge Regression	0.852	−0.019
v4	Gradient Boosted Trees	0.821	−0.050
v5b	Pure BPR Neural Network	0.878	+0.007
v5b	Hybrid (gap > 5)	0.908	+0.037
v5b	Hybrid (gap > 10)	**0.911**	**+0.040**
v6	Dual MF + Ensemble Hybrid	0.907	+0.036

Table 7: Complete progression from heuristic baseline to best score.

The journey from 0.871 to 0.911 involved several lessons:

1. **Verify the ceiling before moving on.** v3 confirmed that no heuristic tuning could break 0.871, justifying the shift to machine learning.

2. **Match the loss function to the metric.** Pointwise regression (v4) optimizes prediction accuracy, not ranking quality. BPR loss (v5) directly optimizes pairwise ranking, which is what AUC-ROC measures.

3. **Train on the test scenario.** Training on groups of six candidates, exactly matching the test format, was critical to v5's success over v4's arbitrary-pair training.

4. **Combine complementary systems.** The hybrid strategy was the single biggest improvement (+0.033 from pure model to hybrid at gap 5). The heuristic and model have different failure modes, and routing each user to the more reliable system yields a large gain.

5. **Simpler models can generalize better.** v6 had higher validation AUC but lower Kaggle AUC than v5b. More features, more models, and more complexity introduced overfitting to the training distribution.

6. **Threshold tuning is free performance.** Sweeping the hybrid confidence threshold from 5 to 10 gained +0.003 with zero additional training, simply by trusting the model on more borderline users.

A Python Source Code

A.1 v3: Conservative Heuristic Tuning

```python
"""
EE627A Midterm v3 - Conservative Improvements over v2
Jude Eschete

v2 baseline (0.871): has_album*album + has_artist*artist + genre_count*genre_mean/10

Lessons from failed v3 attempts:
  - User normalization HURTS on real test set (0.809-0.868)
  - Ensemble min-max normalization destroys raw signal (0.850)
  - The v2 formula is already strong; only small additive changes are safe

This version tries ONLY conservative modifications:
  1. Fine-grained weight grid search (narrow range around v2's 1.0/1.0/0.1)
  2. Small interaction bonus when album+artist signals agree
  3. genre_max as alternative to genre_mean
  4. Confidence multiplier (small boost when more evidence exists)
  5. Additive raw ensemble (sum of raw scores, no normalization)
"""

import os
import math
import random
from collections import defaultdict
from itertools import product

# ======================================================================
# Configuration
# ======================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "..", "Homework 4", "Data")

TRAIN_FILE = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE = os.path.join(DATA_DIR, "albumData2.txt")

OUTPUT_DIR = SCRIPT_DIR


# ======================================================================
# 1. Data Parsing
```

```python
# ======================================================================

def parse_training(path):
    print("[1/4] Loading training data ...")
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"       {len(users):,} users, {total:,} ratings")
    return users


def parse_tracks(path):
    print("[2/4] Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"       {len(tracks):,} tracks")
    return tracks


def parse_albums(path):
    print("[3/4] Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
```

```
            art = None if len(parts) < 2 or parts[1] == "None" else int(parts[1])
            genres = []
            for g in parts[2:]:
                g = g.strip()
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    print(f"       {len(albums):,} albums")
    return albums


def parse_test(path):
    print("[4/4] Loading test data ...")
    test = []
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    print(f"       {len(test):,} test users, "
          f"{sum(len(t) for _, t in test):,} candidates")
    return test


# ======================================================================
# 2. Album Enrichment
# ======================================================================

def enrich_tracks_with_albums(tracks, albums):
    enriched = {}
    stats = {"artist_filled": 0, "genres_added": 0, "tracks_enriched": 0}
    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
```

```
                new_art = album_art
                stats["artist_filled"] += 1
            track_genre_set = set(genres)
            added = 0
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    added += 1
            if added > 0:
                stats["genres_added"] += added
                stats["tracks_enriched"] += 1
        enriched[tid] = (alb, new_art, new_genres)
    print(f"  Album Enrichment: {stats['artist_filled']:,} artists filled, "
          f"{stats['tracks_enriched']:,} tracks got new genres")
    return enriched


# ======================================================================
# 3. Feature Engineering
# ======================================================================

def compute_features(user_ratings, track_meta, uid, tid):
    """Build feature vector for one (user, candidate-track) pair."""
    ratings = user_ratings.get(uid, {})
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    # Album score
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = ratings[alb_id], 1
    else:
        album_score, has_album = 0, 0

    # Artist score
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = ratings[art_id], 1
    else:
        artist_score, has_artist = 0, 0

    # Genre statistics
    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    n = len(genre_scores)
    if n > 0:
        g_max = max(genre_scores)
        g_min = min(genre_scores)
        g_mean = sum(genre_scores) / n
```

```
        g_var = sum((s - g_mean) ** 2 for s in genre_scores) / n
    else:
        g_max = g_min = 0
        g_mean = g_var = 0.0

    # Evidence count
    evidence_count = has_album + has_artist + (1 if n > 0 else 0)

    # Interaction (only when both signals present)
    album_artist_interact = (album_score * artist_score / 100.0
                             if (has_album and has_artist) else 0.0)

    return {
        "album_score": album_score,
        "has_album": has_album,
        "artist_score": artist_score,
        "has_artist": has_artist,
        "genre_count": n,
        "genre_max": g_max,
        "genre_min": g_min,
        "genre_mean": g_mean,
        "genre_var": g_var,
        "evidence_count": evidence_count,
        "album_artist_interact": album_artist_interact,
    }


# ======================================================================
# 4. Scoring Strategies (all conservative v2 variants)
# ======================================================================

def strategy_v2_evidence(feats_list):
    """Original v2: has_album*album + has_artist*artist + genre_count*
        genre_mean/10
    Kaggle: 0.871"""
    return [
        f["has_album"] * f["album_score"]
        + f["has_artist"] * f["artist_score"]
        + f["genre_count"] * f["genre_mean"] / 10.0
        for f in feats_list
    ]


def strategy_tunable(feats_list, wa=1.0, war=1.0, wg=0.1):
    """v2 with tunable weights."""
    return [
        wa * f["has_album"] * f["album_score"]
```

```
        + war * f["has_artist"] * f["artist_score"]
        + wg * f["genre_count"] * f["genre_mean"]
        for f in feats_list
    ]


def strategy_v2_plus_interaction(feats_list, wi=0.1):
    """v2 + small bonus when album AND artist signals agree."""
    return [
        f["has_album"] * f["album_score"]
        + f["has_artist"] * f["artist_score"]
        + f["genre_count"] * f["genre_mean"] / 10.0
        + wi * f["album_artist_interact"]
        for f in feats_list
    ]


def strategy_v2_genre_max(feats_list):
    """v2 but using genre_max instead of genre_mean."""
    return [
        f["has_album"] * f["album_score"]
        + f["has_artist"] * f["artist_score"]
        + (1 if f["genre_count"] > 0 else 0) * f["genre_max"] / 5.0
        for f in feats_list
    ]


def strategy_v2_genre_both(feats_list):
    """v2 with both genre_mean and genre_max."""
    return [
        f["has_album"] * f["album_score"]
        + f["has_artist"] * f["artist_score"]
        + f["genre_count"] * f["genre_mean"] / 10.0
        + (1 if f["genre_count"] > 0 else 0) * f["genre_max"] / 20.0
        for f in feats_list
    ]


def strategy_confidence(feats_list):
    """v2 * (1 + small evidence bonus). Rewards candidates with more data.
    """
    return [
        (f["has_album"] * f["album_score"]
         + f["has_artist"] * f["artist_score"]
         + f["genre_count"] * f["genre_mean"] / 10.0)
        * (1.0 + 0.05 * f["evidence_count"])
        for f in feats_list
```

```python
    ]


def strategy_additive_ensemble(feats_list):
    """Sum raw scores from v2 + interaction + confidence (no normalization).
    All three share the same scale, so summing preserves signal."""
    s_v2 = strategy_v2_evidence(feats_list)
    s_int = strategy_v2_plus_interaction(feats_list, wi=0.15)
    s_conf = strategy_confidence(feats_list)
    return [a + b + c for a, b, c in zip(s_v2, s_int, s_conf)]


# ====================================================================
# 5. Ranking & Submission
# ====================================================================

def rank_top3(scores, track_ids):
    paired = sorted(zip(scores, track_ids), key=lambda x: x[0], reverse=True)
    top3 = set(tid for _, tid in paired[:3])
    return {tid: (1 if tid in top3 else 0) for tid in track_ids}


def write_submission(results, path):
    with open(path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")


def run_strategy(user_ratings, track_meta, test_users,
                 strategy_fn, out_path, strategy_name=None):
    name = strategy_name or strategy_fn.__name__
    print(f"\n>>> Running {name} ...")
    results = []
    for uid, candidates in test_users:
        feats = [compute_features(user_ratings, track_meta, uid, tid)
                 for tid in candidates]
        scores = strategy_fn(feats)
        recs = rank_top3(scores, candidates)
        for tid in candidates:
            results.append((f"{uid}_{tid}", recs[tid]))
    write_submission(results, out_path)
    ones = sum(v for _, v in results)
    print(f"    {len(results):,} predictions ({ones:,} rec, {len(results)-ones:,} don't)")
```

```python
    print(f"    Saved -> {out_path}")
    return results


# ======================================================================
# 6. Validation & Grid Search
# ======================================================================

def build_validation_set(user_ratings, track_meta, fraction=0.1, seed=42):
    """Hold out tracks from 10% of users. Mimics test scenario."""
    rng = random.Random(seed)
    track_ids = set(track_meta.keys())
    all_uids = list(user_ratings.keys())
    rng.shuffle(all_uids)
    n_val = int(len(all_uids) * fraction)

    val_set = []
    val_user_held_items = {}
    for uid in all_uids[:n_val]:
        user_tracks = [iid for iid in user_ratings[uid] if iid in track_ids]
        if len(user_tracks) < 6:
            continue
        sample = rng.sample(user_tracks, 6)
        sample_ratings = {tid: user_ratings[uid][tid] for tid in sample}
        val_set.append((uid, sample, sample_ratings))
        val_user_held_items[uid] = set(sample)

    train_reduced = {}
    for uid, ratings in user_ratings.items():
        if uid in val_user_held_items:
            held = val_user_held_items[uid]
            train_reduced[uid] = {k: v for k, v in ratings.items()
                                  if k not in held}
        else:
            train_reduced[uid] = ratings

    print(f"  Validation: {len(val_set):,} users, {len(val_set)*6:,} candidates")
    return train_reduced, val_set


def compute_auc(val_set, user_ratings, track_meta, strategy_fn):
    """AUC on validation: do we rank the true top-3 higher than bottom-3?"""
    correct = 0
    total = 0
```

```python
    for uid, items, true_ratings in val_set:
        feats = [compute_features(user_ratings, track_meta, uid, tid)
                 for tid in items]
        scores = strategy_fn(feats)
        rated = sorted(zip([true_ratings[iid] for iid in items], items),
                       reverse=True)
        positives = set(tid for _, tid in rated[:3])
        negatives = set(tid for _, tid in rated[3:])
        score_map = dict(zip(items, scores))
        for p in positives:
            for n in negatives:
                if score_map[p] > score_map[n]:
                    correct += 1
                elif score_map[p] == score_map[n]:
                    correct += 0.5
                total += 1
    return correct / total if total > 0 else 0.0


def grid_search(val_set, user_ratings, track_meta):
    """Fine-grained grid search around the v2 sweet spot (1.0/1.0/0.1)."""
    print("\n  Fine-grained grid search around v2 weights ...")
    best_auc = 0
    best_params = None

    # Narrow ranges centered on v2 defaults
    wa_range = [0.8, 0.9, 1.0, 1.1, 1.2]
    war_range = [0.8, 0.9, 1.0, 1.1, 1.2, 1.3]
    wg_range = [0.06, 0.08, 0.10, 0.12, 0.14]

    total = len(wa_range) * len(war_range) * len(wg_range)
    tested = 0

    for wa, war, wg in product(wa_range, war_range, wg_range):
        def strat(feats, _wa=wa, _war=war, _wg=wg):
            return strategy_tunable(feats, _wa, _war, _wg)
        auc = compute_auc(val_set, user_ratings, track_meta, strat)
        tested += 1
        if auc > best_auc:
            best_auc = auc
            best_params = (wa, war, wg)
            print(f"    [{tested}/{total}] NEW BEST AUC={auc:.4f} "
                  f"album={wa} artist={war} genre={wg}")

    print(f"\n  Best: album={best_params[0]}, artist={best_params[1]}, "
          f"genre={best_params[2]}  AUC={best_auc:.4f}")
    return best_params, best_auc
```

```python


def grid_search_interaction(val_set, user_ratings, track_meta):
    """Search interaction weight added to v2."""
    print("\n  Grid searching interaction weight ...")
    best_auc = 0
    best_wi = 0

    for wi in [0.0, 0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.50]:
        def strat(feats, _wi=wi):
            return strategy_v2_plus_interaction(feats, _wi)
        auc = compute_auc(val_set, user_ratings, track_meta, strat)
        if auc > best_auc:
            best_auc = auc
            best_wi = wi
            print(f"    wi={wi:.2f}  AUC={auc:.4f} *")
        else:
            print(f"    wi={wi:.2f}  AUC={auc:.4f}")

    print(f"\n  Best interaction weight: {best_wi}  AUC={best_auc:.4f}")
    return best_wi, best_auc


# ====================================================================
# 7. Main
# ====================================================================

def main():
    user_ratings = parse_training(TRAIN_FILE)
    track_meta = parse_tracks(TRACK_FILE)
    album_meta = parse_albums(ALBUM_FILE)
    test_users = parse_test(TEST_FILE)

    print("\n" + "=" * 60)
    print("  ENRICHING TRACKS")
    print("=" * 60)
    enriched_meta = enrich_tracks_with_albums(track_meta, album_meta)

    # ---- Validation ----
    print("\n" + "=" * 60)
    print("  VALIDATION & GRID SEARCH")
    print("=" * 60)
    train_reduced, val_set = build_validation_set(user_ratings,
        enriched_meta)

    # Baseline
    v2_auc = compute_auc(val_set, train_reduced, enriched_meta,
```

```
                                strategy_v2_evidence)
    print(f"\n  v2 baseline val AUC: {v2_auc:.4f}")

    # Evaluate all conservative variants
    print("\n  Strategy comparison:")
    for name, fn in [
        ("v2_evidence", strategy_v2_evidence),
        ("v2+interaction(0.1)", lambda f: strategy_v2_plus_interaction(f, 0.1)),
        ("v2+interaction(0.15)", lambda f: strategy_v2_plus_interaction(f, 0.15)),
        ("v2_genre_max", strategy_v2_genre_max),
        ("v2_genre_both", strategy_v2_genre_both),
        ("confidence", strategy_confidence),
        ("additive_ensemble", strategy_additive_ensemble),
    ]:
        auc = compute_auc(val_set, train_reduced, enriched_meta, fn)
        delta = auc - v2_auc
        marker = " <-- BEST" if delta > 0 else ""
        print(f"    {name:30s} AUC={auc:.4f} ({delta:+.4f}){marker}")

    # Grid searches
    best_params, best_tune_auc = grid_search(val_set, train_reduced, enriched_meta)
    best_wi, best_int_auc = grid_search_interaction(val_set, train_reduced, enriched_meta)

    # ---- Generate submissions ----
    print("\n" + "=" * 60)
    print("  GENERATING SUBMISSIONS")
    print("=" * 60)

    # 1: Tuned weights
    wa, war, wg = best_params
    def tuned(feats):
        return strategy_tunable(feats, wa, war, wg)
    run_strategy(user_ratings, enriched_meta, test_users, tuned,
                 os.path.join(OUTPUT_DIR, "submission_v3_tuned.csv"),
                 f"tuned(a={wa},ar={war},g={wg})")

    # 2: v2 + interaction
    def interact(feats):
        return strategy_v2_plus_interaction(feats, best_wi)
    run_strategy(user_ratings, enriched_meta, test_users, interact,
                 os.path.join(OUTPUT_DIR, "submission_v3_interact.csv"),
                 f"v2+interaction(wi={best_wi})")

```

```python
    # 3: Genre both (mean + max)
    run_strategy(user_ratings, enriched_meta, test_users,
                 strategy_v2_genre_both,
                 os.path.join(OUTPUT_DIR, "submission_v3_genre_both.csv"),
                 "v2_genre_both")

    # 4: Confidence
    run_strategy(user_ratings, enriched_meta, test_users,
                 strategy_confidence,
                 os.path.join(OUTPUT_DIR, "submission_v3_confidence.csv"),
                 "confidence")

    # 5: Additive ensemble (raw sum, no normalization)
    run_strategy(user_ratings, enriched_meta, test_users,
                 strategy_additive_ensemble,
                 os.path.join(OUTPUT_DIR, "submission_v3_additive.csv"),
                 "additive_ensemble")

    # ---- Summary ----
    print("\n" + "=" * 60)
    print("  RESULTS")
    print("=" * 60)
    print(f"  v2 baseline val AUC:   {v2_auc:.4f}  (Kaggle: 0.871)")
    print(f"  tuned val AUC:         {best_tune_auc:.4f}")
    print(f"  interaction val AUC:   {best_int_auc:.4f}")
    print()
    print("  Files to upload:")
    print(f"    submission_v3_tuned.csv       (alb={wa}, art={war}, g={wg
        })")
    print(f"    submission_v3_interact.csv    (wi={best_wi})")
    print(f"    submission_v3_genre_both.csv")
    print(f"    submission_v3_confidence.csv")
    print(f"    submission_v3_additive.csv")
    print("\nDone.")


if __name__ == "__main__":
    main()
```

Listing 1: EE627_Midterm_v3_improved.py

A.2 v4: Pointwise ML Models

```python
"""
EE627A Midterm v4 - PyTorch Pairwise Ranking Model (GPU)
Jude Eschete

Key insight from failed attempts:
  - Heuristic weight tuning hit ceiling at 0.871
  - Pointwise regression (predict rating) doesn't optimize ranking (0.821-0.852)
  - Pairwise ranking loss directly optimizes "which candidate ranks higher"
    which is exactly what AUC measures

Approach:
  1. Build (user, track) feature vectors from album/artist/genre signals
  2. Construct PAIRS: (preferred_track, non-preferred_track) from user history
  3. Train a neural network with pairwise margin ranking loss:
     loss = max(0, margin - (score(preferred) - score(non-preferred)))
  4. The model learns a scoring function that produces correct relative
     rankings, not absolute ratings
  5. At test time, score 6 candidates, pick top 3

Uses RTX 5080 for fast training.
"""

import os
import sys
import random
import math
import time
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# ======================================================================
# Configuration
# ======================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "..", "Homework 4", "Data")

TRAIN_FILE = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE = os.path.join(DATA_DIR, "albumData2.txt")
```

```python

OUTPUT_DIR = SCRIPT_DIR
RESULTS_FILE = os.path.join(SCRIPT_DIR, "results.txt")
SEED = 42
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")


class Tee:
    """Write to both stdout and a file simultaneously."""
    def __init__(self, filepath):
        self.file = open(filepath, "w")
        self.stdout = sys.stdout

    def write(self, data):
        self.stdout.write(data)
        self.file.write(data)

    def flush(self):
        self.stdout.flush()
        self.file.flush()

    def close(self):
        self.file.close()
        sys.stdout = self.stdout


# ======================================================================
# 1. Data Parsing
# ======================================================================

def parse_training(path):
    print("[1/4] Loading training data ...")
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"       {len(users):,} users, {total:,} ratings")
    return users

```

```python

def parse_tracks(path):
    print("[2/4] Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"       {len(tracks):,} tracks")
    return tracks


def parse_albums(path):
    print("[3/4] Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
            art = None if len(parts) < 2 or parts[1] == "None" else int(parts[1])
            genres = []
            for g in parts[2:]:
                g = g.strip()
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    print(f"       {len(albums):,} albums")
    return albums


def parse_test(path):
    print("[4/4] Loading test data ...")
    test = []
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
```

```
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    print(f"       {len(test):,} test users, "
          f"{sum(len(t) for _, t in test):,} candidates")
    return test


# ======================================================================
# 2. Album Enrichment
# ======================================================================

def enrich_tracks_with_albums(tracks, albums):
    enriched = {}
    stats = {"artist_filled": 0, "genres_added": 0}
    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
                new_art = album_art
                stats["artist_filled"] += 1
            track_genre_set = set(genres)
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    stats["genres_added"] += 1
        enriched[tid] = (alb, new_art, new_genres)
    print(f"  Enrichment: {stats['artist_filled']:,} artists filled, "
          f"{stats['genres_added']:,} genres added")
    return enriched


# ======================================================================
# 3. Feature Engineering
# ======================================================================

N_FEATURES = 19
```

```python

def compute_user_profile(ratings):
    vals = list(ratings.values())
    n = len(vals)
    if n == 0:
        return 0.0, 0.0, 0
    mean = sum(vals) / n
    var = sum((v - mean) ** 2 for v in vals) / n if n > 1 else 0.0
    return mean, var ** 0.5, n


def compute_feature_vec(ratings, track_meta, tid, u_mean, u_std, u_n):
    """19-dimensional feature vector."""
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    if alb_id is not None and alb_id in ratings:
        album_score, has_album = float(ratings[alb_id]), 1.0
    else:
        album_score, has_album = 0.0, 0.0

    if art_id is not None and art_id in ratings:
        artist_score, has_artist = float(ratings[art_id]), 1.0
    else:
        artist_score, has_artist = 0.0, 0.0

    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    gc = len(genre_scores)
    if gc > 0:
        g_max = float(max(genre_scores))
        g_min = float(min(genre_scores))
        g_mean = sum(genre_scores) / gc
        g_var = sum((s - g_mean) ** 2 for s in genre_scores) / gc
    else:
        g_max = g_min = 0.0
        g_mean = g_var = 0.0

    evidence_count = has_album + has_artist + (1.0 if gc > 0 else 0.0)

    aa_int = album_score * artist_score / 100.0 if (has_album and has_artist) else 0.0
    ga_int = g_mean * artist_score / 100.0 if (has_artist and gc > 0) else 0.0
    ag_int = album_score * g_mean / 100.0 if (has_album and gc > 0) else 0.0

    album_above = (album_score - u_mean) if has_album else 0.0
    artist_above = (artist_score - u_mean) if has_artist else 0.0
```

```
    genre_above = (g_mean - u_mean) if gc > 0 else 0.0

    return [
        album_score, has_album,
        artist_score, has_artist,
        float(gc), g_max, g_min, g_mean, g_var,
        evidence_count,
        aa_int, ga_int, ag_int,
        u_mean, u_std, float(u_n),
        album_above, artist_above, genre_above,
    ]


# ======================================================================
# 4. Pairwise Training Data
# ======================================================================

class PairwiseDataset(Dataset):
    """Dataset of (preferred_features, non_preferred_features) pairs."""
    def __init__(self, X_pos, X_neg):
        self.X_pos = torch.tensor(X_pos, dtype=torch.float32)
        self.X_neg = torch.tensor(X_neg, dtype=torch.float32)

    def __len__(self):
        return len(self.X_pos)

    def __getitem__(self, idx):
        return self.X_pos[idx], self.X_neg[idx]


def build_pairwise_data(user_ratings, track_meta, pairs_per_user=100,
                        seed=42):
    """Build pairwise training data from user track ratings.

    For each user, sample pairs of tracks where one was rated higher.
    The model learns: score(higher_rated) > score(lower_rated).
    """
    print("\n  Building pairwise training data ...")
    rng = random.Random(seed)
    track_ids = set(track_meta.keys())

    X_pos_rows = []
    X_neg_rows = []
    users_used = 0

    for uid, ratings in user_ratings.items():
```

```
        user_tracks = [(iid, ratings[iid]) for iid in ratings if iid in track_ids]
        if len(user_tracks) < 2:
            continue

        users_used += 1
        u_mean, u_std, u_n = compute_user_profile(ratings)

        # Cache features for this user's tracks
        track_feats = {}
        for tid, _ in user_tracks:
            track_feats[tid] = compute_feature_vec(
                ratings, track_meta, tid, u_mean, u_std, u_n)

        # Sample pairs: pick two tracks, higher-rated = positive
        n_pairs = min(pairs_per_user, len(user_tracks) * (len(user_tracks) - 1) // 2)
        for _ in range(n_pairs):
            t1, t2 = rng.sample(user_tracks, 2)
            tid1, r1 = t1
            tid2, r2 = t2
            if r1 == r2:
                continue  # Skip ties
            if r1 > r2:
                X_pos_rows.append(track_feats[tid1])
                X_neg_rows.append(track_feats[tid2])
            else:
                X_pos_rows.append(track_feats[tid2])
                X_neg_rows.append(track_feats[tid1])

    X_pos = np.array(X_pos_rows, dtype=np.float32)
    X_neg = np.array(X_neg_rows, dtype=np.float32)
    print(f"       {users_used:,} users, {len(X_pos):,} pairs")
    return X_pos, X_neg


# ====================================================================
# 5. Neural Network
# ====================================================================

class RankingNet(nn.Module):
    """MLP that scores a (user, track) feature vector.
    Trained with pairwise margin loss so it learns relative rankings."""

    def __init__(self, n_features, hidden_sizes=(128, 64, 32)):
        super().__init__()
        layers = []
```

```python
        prev = n_features
        for h in hidden_sizes:
            layers.append(nn.Linear(prev, h))
            layers.append(nn.BatchNorm1d(h))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(0.2))
            prev = h
        layers.append(nn.Linear(prev, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x).squeeze(-1)


def train_model(X_pos, X_neg, n_epochs=30, batch_size=4096, lr=1e-3,
                margin=1.0):
    """Train with MarginRankingLoss on GPU."""
    print(f"\n  Training on {DEVICE} ...")

    dataset = PairwiseDataset(X_pos, X_neg)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
                        num_workers=0, pin_memory=True)

    model = RankingNet(N_FEATURES).to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-5)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, n_epochs)
    criterion = nn.MarginRankingLoss(margin=margin)

    target = torch.ones(batch_size, device=DEVICE)  # pos should rank higher

    for epoch in range(n_epochs):
        model.train()
        total_loss = 0
        n_batches = 0

        for x_pos, x_neg in loader:
            x_pos = x_pos.to(DEVICE)
            x_neg = x_neg.to(DEVICE)
            bs = x_pos.size(0)

            s_pos = model(x_pos)
            s_neg = model(x_neg)

            t = target[:bs]
```

```
            loss = criterion(s_pos, s_neg, t)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            n_batches += 1

        scheduler.step()
        avg_loss = total_loss / n_batches

        # Compute accuracy: how often does model rank positive > negative
        if (epoch + 1) % 5 == 0 or epoch == 0:
            model.eval()
            correct = 0
            total = 0
            with torch.no_grad():
                for x_pos, x_neg in loader:
                    x_pos = x_pos.to(DEVICE)
                    x_neg = x_neg.to(DEVICE)
                    s_pos = model(x_pos)
                    s_neg = model(x_neg)
                    correct += (s_pos > s_neg).sum().item()
                    total += x_pos.size(0)
            acc = correct / total
            print(f"    Epoch {epoch+1:3d}/{n_epochs}  "
                  f"loss={avg_loss:.4f}  pair_acc={acc:.4f}")

    return model


# ======================================================================
# 6. Validation
# ======================================================================

def validate(model, user_ratings, track_meta, fraction=0.1, seed=42):
    """AUC validation: can model rank top-3 tracks above bottom-3?"""
    print("\n  Validating ...")
    model.eval()
    rng = random.Random(seed)
    track_ids = set(track_meta.keys())

    all_uids = list(user_ratings.keys())
    rng.shuffle(all_uids)
    n_val = int(len(all_uids) * fraction)

```

```
    correct = 0
    total = 0
    val_users = 0

    with torch.no_grad():
        for uid in all_uids[:n_val]:
            ratings = user_ratings[uid]
            user_tracks = [iid for iid in ratings if iid in track_ids]
            if len(user_tracks) < 6:
                continue

            sample = rng.sample(user_tracks, 6)
            true_ratings = {tid: ratings[tid] for tid in sample}
            u_mean, u_std, u_n = compute_user_profile(ratings)

            X = torch.tensor(
                [compute_feature_vec(ratings, track_meta, tid, u_mean,
                    u_std, u_n)
                 for tid in sample],
                dtype=torch.float32, device=DEVICE
            )
            scores = model(X).cpu().numpy()

            rated = sorted(zip([true_ratings[t] for t in sample], sample),
                           reverse=True)
            positives = set(tid for _, tid in rated[:3])
            negatives = set(tid for _, tid in rated[3:])
            score_map = dict(zip(sample, scores))

            for p in positives:
                for n in negatives:
                    if score_map[p] > score_map[n]:
                        correct += 1
                    elif score_map[p] == score_map[n]:
                        correct += 0.5
                    total += 1
            val_users += 1

    auc = correct / total if total > 0 else 0.0
    print(f"       {val_users:,} val users, AUC = {auc:.4f}")
    return auc


def validate_v2(user_ratings, track_meta, fraction=0.1, seed=42):
    """v2 heuristic baseline for comparison."""
    print("\n  Validating v2 heuristic ...")
    rng = random.Random(seed)
```

```
    track_ids = set(track_meta.keys())
    all_uids = list(user_ratings.keys())
    rng.shuffle(all_uids)
    n_val = int(len(all_uids) * fraction)

    correct = 0
    total = 0
    val_users = 0

    for uid in all_uids[:n_val]:
        ratings = user_ratings[uid]
        user_tracks = [iid for iid in ratings if iid in track_ids]
        if len(user_tracks) < 6:
            continue
        sample = rng.sample(user_tracks, 6)
        true_ratings = {tid: ratings[tid] for tid in sample}
        scores = []
        for tid in sample:
            alb_id, art_id, genre_ids = track_meta.get(tid, (None, None,
                []))
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            scores.append(s)
        rated = sorted(zip([true_ratings[t] for t in sample], sample),
                       reverse=True)
        positives = set(tid for _, tid in rated[:3])
        negatives = set(tid for _, tid in rated[3:])
        score_map = dict(zip(sample, scores))
        for p in positives:
            for n in negatives:
                if score_map[p] > score_map[n]:
                    correct += 1
                elif score_map[p] == score_map[n]:
                    correct += 0.5
                total += 1
        val_users += 1

    auc = correct / total if total > 0 else 0.0
    print(f"       {val_users:,} val users, AUC = {auc:.4f}")
    return auc

```

```python

# ======================================================================
# 7. Prediction
# ======================================================================

def predict_and_submit(model, user_ratings, track_meta, test_users,
                       out_path, model_name="model"):
    print(f"\n>>> Predicting with {model_name} ...")
    model.eval()
    results = []

    with torch.no_grad():
        for uid, candidates in test_users:
            ratings = user_ratings.get(uid, {})
            u_mean, u_std, u_n = compute_user_profile(ratings)

            X = torch.tensor(
                [compute_feature_vec(ratings, track_meta, tid, u_mean,
                   u_std, u_n)
                 for tid in candidates],
                dtype=torch.float32, device=DEVICE
            )
            scores = model(X).cpu().numpy()

            ranked = sorted(zip(scores, candidates), reverse=True)
            top3 = set(tid for _, tid in ranked[:3])
            for tid in candidates:
                results.append((f"{uid}_{tid}", 1 if tid in top3 else 0))

    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")

    ones = sum(v for _, v in results)
    print(f"    {len(results):,} predictions ({ones:,} rec, "
          f"{len(results)-ones:,} don't)")
    print(f"    Saved -> {out_path}")


# ======================================================================
# 8. Main
# ======================================================================

def main():
    tee = Tee(RESULTS_FILE)
    sys.stdout = tee
```

```
    start_time = time.time()

    torch.manual_seed(SEED)
    random.seed(SEED)
    np.random.seed(SEED)

    print(f"  Device: {DEVICE}")
    if torch.cuda.is_available():
        print(f"  GPU: {torch.cuda.get_device_name(0)}")

    user_ratings = parse_training(TRAIN_FILE)
    track_meta = parse_tracks(TRACK_FILE)
    album_meta = parse_albums(ALBUM_FILE)
    test_users = parse_test(TEST_FILE)

    print("\n" + "=" * 60)
    print("  ENRICHING TRACKS")
    print("=" * 60)
    enriched_meta = enrich_tracks_with_albums(track_meta, album_meta)

    # ---- Build pairwise training data ----
    print("\n" + "=" * 60)
    print("  BUILDING PAIRWISE TRAINING DATA")
    print("=" * 60)
    X_pos, X_neg = build_pairwise_data(user_ratings, enriched_meta,
                                       pairs_per_user=100)

    # ---- Train model ----
    print("\n" + "=" * 60)
    print("  TRAINING RANKING MODEL")
    print("=" * 60)
    model = train_model(X_pos, X_neg, n_epochs=30, batch_size=4096,
                        lr=1e-3, margin=1.0)

    # ---- Validation ----
    print("\n" + "=" * 60)
    print("  VALIDATION")
    print("=" * 60)
    v2_auc = validate_v2(user_ratings, enriched_meta)
    nn_auc = validate(model, user_ratings, enriched_meta)

    print(f"\n  v2 heuristic:    {v2_auc:.4f}  (Kaggle: 0.871)")
    print(f"  Pairwise NN:     {nn_auc:.4f}")

    # ---- Also train a wider/deeper variant ----
    print("\n" + "=" * 60)
    print("  TRAINING LARGER MODEL")
```

```
    print("=" * 60)
    model_large = train_model(X_pos, X_neg, n_epochs=50, batch_size=8192,
                              lr=5e-4, margin=0.5)
    nn_large_auc = validate(model_large, user_ratings, enriched_meta)
    print(f"  Larger model:    {nn_large_auc:.4f}")

    # ---- Submissions ----
    print("\n" + "=" * 60)
    print("  GENERATING SUBMISSIONS")
    print("=" * 60)

    predict_and_submit(model, user_ratings, enriched_meta, test_users,
                       os.path.join(OUTPUT_DIR, "submission_v4_pairwise.csv"),
                       "Pairwise NN")

    predict_and_submit(model_large, user_ratings, enriched_meta, test_users,
                       os.path.join(OUTPUT_DIR, "submission_v4_pairwise_lg.csv"),
                       "Pairwise NN (large)")

    # ---- Summary ----
    print("\n" + "=" * 60)
    print("  RESULTS")
    print("=" * 60)
    print(f"  v2 heuristic val AUC:     {v2_auc:.4f}  (Kaggle: 0.871)")
    print(f"  Pairwise NN val AUC:      {nn_auc:.4f}")
    print(f"  Pairwise NN (lg) val AUC: {nn_large_auc:.4f}")
    print()
    print("  Files:")
    print("    submission_v4_pairwise.csv     - Pairwise ranking NN")
    print("    submission_v4_pairwise_lg.csv  - Larger pairwise NN")

    elapsed = time.time() - start_time
    print(f"\n  Total time: {elapsed:.1f}s ({elapsed/60:.1f}m)")
    print("\nDone.")

    tee.close()


if __name__ == "__main__":
    main()
```

Listing 2: EE627_Midterm_v4_model.py

A.3 v5: BPR Neural Network with Matrix Factorization

```python
"""
EE627A Midterm v5 - Full Model with All Improvements
Jude Eschete

10 improvements over v2 heuristic (0.871) and v4 collapsed model:

  1. Train on groups of 6 (mimics test scenario exactly)
  2. BPR pairwise loss (stable, no collapse like MarginRankingLoss)
  3. Heavy regularization (small net, dropout, weight decay, early stop)
  4. Feature normalization (StandardScaler on inputs)
  5. Collaborative filtering features (sibling track ratings)
  6. Popularity/rarity features (n_raters, global avg per item)
  7. Cross-candidate features (shared artist/album/genre within group)
  8. Global item statistics as features (not imputation)
  9. Hard negative weighting (close-rating pairs weighted more)
  10. Matrix factorization embeddings (SGD on GPU, dot product feature)

Architecture:
  - 32 features per (user, track, group) pair
  - Small MLP: 32 -> 64 -> 32 -> 1 with BatchNorm + Dropout
  - BPR loss on (top-3, bottom-3) pairs within each 6-candidate group
  - Early stopping on held-out validation AUC

Uses RTX 5080 GPU for MF training and ranking model training.
"""

import os
import sys
import random
import math
import time
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, TensorDataset
from collections import defaultdict

# ======================================================================
# Configuration
# ======================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "Data", "music-recommender-2026s")

OUTPUT_DIR = SCRIPT_DIR
RESULTS_FILE = os.path.join(SCRIPT_DIR, "results.txt")
```

```
SEED = 42
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Hyperparameters (v5b: tuned up from v5a which scored 0.878)
MF_FACTORS = 64          # was 32 -- richer embeddings
MF_EPOCHS = 40           # was 20 -- let MF converge more
MF_LR = 0.005            # was 0.01 -- slower for stability with more
    epochs
MF_BATCH = 65536

GROUPS_PER_USER = 30     # was 15 -- more training data
HIDDEN_SIZES = (128, 64, 32)  # was (64, 32) -- deeper network
DROPOUT = 0.3
LR = 5e-4                # was 1e-3 -- slower learning for deeper net
WEIGHT_DECAY = 1e-4
BATCH_SIZE = 2048
MAX_EPOCHS = 150          # was 100 -- more room with early stopping
PATIENCE = 15             # was 10 -- more patience for deeper net
HARD_NEG_TEMP = 15.0      # was 20 -- focus more on hard pairs

N_BASE_FEATURES = 29   # per (user, track)
N_CROSS_FEATURES = 3   # per (user, track, group)
N_FEATURES = N_BASE_FEATURES + N_CROSS_FEATURES  # 32 total


# ======================================================================
# Logging
# ======================================================================

class Tee:
    def __init__(self, filepath):
        self.file = open(filepath, "w", encoding="utf-8")
        self.stdout = sys.stdout

    def write(self, data):
        self.stdout.write(data)
        self.file.write(data)

    def flush(self):
        self.stdout.flush()
        self.file.flush()

    def close(self):
        self.file.close()
        sys.stdout = self.stdout


```

```python
# ======================================================================
# 1. Data Parsing
# ======================================================================

def parse_training(path):
    print("[1/4] Loading training data ...")
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"       {len(users):,} users, {total:,} ratings")
    return users


def parse_tracks(path):
    print("[2/4] Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"       {len(tracks):,} tracks")
    return tracks


def parse_albums(path):
    print("[3/4] Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
```

```
                continue
            alb_id = int(parts[0])
            art = None if len(parts) < 2 or parts[1] == "None" else int(
                parts[1])
            genres = []
            for g in parts[2:]:
                g = g.strip()
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    print(f"       {len(albums):,} albums")
    return albums


def parse_test(path):
    print("[4/4] Loading test data ...")
    test = []
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    print(f"       {len(test):,} test users, "
          f"{sum(len(t) for _, t in test):,} candidates")
    return test


# ====================================================================
# 2. Album Enrichment
# ====================================================================

def enrich_tracks_with_albums(tracks, albums):
    enriched = {}
    stats = {"artist_filled": 0, "genres_added": 0}
    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
```

```
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
                new_art = album_art
                stats["artist_filled"] += 1
            track_genre_set = set(genres)
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    stats["genres_added"] += 1
        enriched[tid] = (alb, new_art, new_genres)
    print(f"  Enrichment: {stats['artist_filled']:,} artists filled, "
          f"{stats['genres_added']:,} genres added")
    return enriched


# ======================================================================
# 3. Auxiliary Data Structures
# ======================================================================

def build_global_item_stats(user_ratings):
    """Per-item: (mean_rating, n_raters)."""
    print("  Building global item stats ...")
    item_sum = defaultdict(float)
    item_cnt = defaultdict(int)
    for ratings in user_ratings.values():
        for iid, r in ratings.items():
            item_sum[iid] += r
            item_cnt[iid] += 1
    stats = {iid: (item_sum[iid] / item_cnt[iid], item_cnt[iid])
             for iid in item_sum}
    print(f"       {len(stats):,} items with global stats")
    return stats


def build_track_mappings(track_meta):
    """album_id -> [track_ids], artist_id -> [track_ids]."""
    print("  Building track mappings ...")
    album_to_tracks = defaultdict(list)
    artist_to_tracks = defaultdict(list)
    for tid, (alb, art, genres) in track_meta.items():
        if alb is not None:
            album_to_tracks[alb].append(tid)
        if art is not None:
            artist_to_tracks[art].append(tid)
    print(f"       {len(album_to_tracks):,} albums, "
          f"{len(artist_to_tracks):,} artists mapped")
```

```
    return dict(album_to_tracks), dict(artist_to_tracks)


def train_mf_embeddings(user_ratings, n_factors=32, n_epochs=20, lr=0.01,
                        batch_size=65536):
    """Train matrix factorization via SGD on GPU (#10)."""
    print(f"\n  Training MF embeddings (k={n_factors}) on {DEVICE} ...")

    uid_to_idx = {}
    iid_to_idx = {}
    u_list, i_list, r_list = [], [], []

    for uid, ratings in user_ratings.items():
        if uid not in uid_to_idx:
            uid_to_idx[uid] = len(uid_to_idx)
        u_idx = uid_to_idx[uid]
        for iid, r in ratings.items():
            if iid not in iid_to_idx:
                iid_to_idx[iid] = len(iid_to_idx)
            u_list.append(u_idx)
            i_list.append(iid_to_idx[iid])
            r_list.append(float(r))

    n_users = len(uid_to_idx)
    n_items = len(iid_to_idx)
    print(f"       {n_users:,} users, {n_items:,} items, "
          f"{len(r_list):,} ratings")

    users_t = torch.tensor(u_list, dtype=torch.long)
    items_t = torch.tensor(i_list, dtype=torch.long)
    ratings_t = torch.tensor(r_list, dtype=torch.float32)

    dataset = TensorDataset(users_t, items_t, ratings_t)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
                        num_workers=0, pin_memory=(DEVICE.type == "cuda"))

    user_emb = nn.Embedding(n_users, n_factors).to(DEVICE)
    item_emb = nn.Embedding(n_items, n_factors).to(DEVICE)
    nn.init.normal_(user_emb.weight, 0, 0.1)
    nn.init.normal_(item_emb.weight, 0, 0.1)

    optimizer = torch.optim.Adam(
        list(user_emb.parameters()) + list(item_emb.parameters()),
        lr=lr, weight_decay=1e-4
    )

    for epoch in range(n_epochs):
```

```
        total_loss = 0
        n_b = 0
        for u_b, i_b, r_b in loader:
            u_b, i_b, r_b = u_b.to(DEVICE), i_b.to(DEVICE), r_b.to(DEVICE)
            pred = (user_emb(u_b) * item_emb(i_b)).sum(dim=1)
            loss = ((pred - r_b) ** 2).mean()
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            n_b += 1
        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(f"    MF Epoch {epoch+1:3d}/{n_epochs}  "
                  f"MSE={total_loss/n_b:.2f}")

    uf = user_emb.weight.detach().cpu().numpy()
    itf = item_emb.weight.detach().cpu().numpy()
    print(f"       MF done. User factors: {uf.shape}, Item factors: {itf.shape}")
    return uid_to_idx, iid_to_idx, uf, itf


# ======================================================================
# 4. Feature Engineering
# ======================================================================

def compute_user_profile(ratings):
    vals = list(ratings.values())
    n = len(vals)
    if n == 0:
        return 0.0, 0.0, 0
    mean = sum(vals) / n
    var = sum((v - mean) ** 2 for v in vals) / n if n > 1 else 0.0
    return mean, var ** 0.5, n


def compute_base_features(ratings, track_meta, tid, u_mean, u_std, u_n,
                          global_stats, album_to_tracks, artist_to_tracks,
                          mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
                          uid):
    """29 base features per (user, track) pair."""
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    # --- Album ---
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = float(ratings[alb_id]), 1.0
```

```
    else:
        album_score, has_album = 0.0, 0.0

    # --- Artist ---
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = float(ratings[art_id]), 1.0
    else:
        artist_score, has_artist = 0.0, 0.0

    # --- Genre stats ---
    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    gc = len(genre_scores)
    if gc > 0:
        g_max = float(max(genre_scores))
        g_min = float(min(genre_scores))
        g_mean = sum(genre_scores) / gc
        g_var = sum((s - g_mean) ** 2 for s in genre_scores) / gc
    else:
        g_max = g_min = g_mean = g_var = 0.0

    evidence_count = has_album + has_artist + (1.0 if gc > 0 else 0.0)

    # --- Interactions ---
    aa_int = album_score * artist_score / 100.0 if (has_album and
        has_artist) else 0.0
    ga_int = g_mean * artist_score / 100.0 if (has_artist and gc > 0) else
        0.0
    ag_int = album_score * g_mean / 100.0 if (has_album and gc > 0) else
        0.0

    # --- User-relative ---
    album_above = (album_score - u_mean) if has_album else 0.0
    artist_above = (artist_score - u_mean) if has_artist else 0.0
    genre_above = (g_mean - u_mean) if gc > 0 else 0.0

    # --- (#6) Popularity & global avg ---
    alb_gavg, alb_nrat = global_stats.get(alb_id, (0.0, 0)) if alb_id else
        (0.0, 0)
    art_gavg, art_nrat = global_stats.get(art_id, (0.0, 0)) if art_id else
        (0.0, 0)
    g_gavg_sum, g_nrat_sum, g_items = 0.0, 0.0, 0
    for gid in genre_ids:
        if gid in global_stats:
            gavg, nrat = global_stats[gid]
            g_gavg_sum += gavg
            g_nrat_sum += nrat
            g_items += 1
```

```python
    avg_genre_gavg = g_gavg_sum / g_items if g_items > 0 else 0.0
    avg_genre_nrat = g_nrat_sum / g_items if g_items > 0 else 0.0

    # --- (#5) CF sibling features ---
    sibling_ratings = []
    if alb_id is not None:
        for sib in album_to_tracks.get(alb_id, []):
            if sib != tid and sib in ratings:
                sibling_ratings.append(ratings[sib])
    if art_id is not None:
        album_sibs = set(album_to_tracks.get(alb_id, [])) if alb_id else set()
        for sib in artist_to_tracks.get(art_id, []):
            if sib != tid and sib not in album_sibs and sib in ratings:
                sibling_ratings.append(ratings[sib])
    cf_score = sum(sibling_ratings) / len(sibling_ratings) if sibling_ratings else 0.0
    cf_count = len(sibling_ratings)

    # --- (#10) MF dot product ---
    mf_score = 0.0
    if uid in mf_uid_map and tid in mf_iid_map:
        mf_score = float(
            mf_user_f[mf_uid_map[uid]] @ mf_item_f[mf_iid_map[tid]])

    # --- v2 heuristic score as feature ---
    v2_score = (has_album * album_score + has_artist * artist_score
                + gc * g_mean / 10.0)

    return [
        album_score, has_album,                          # 0-1
        artist_score, has_artist,                        # 2-3
        float(gc), g_max, g_min, g_mean, g_var,         # 4-8
        evidence_count,                                  # 9
        aa_int, ga_int, ag_int,                          # 10-12
        u_mean, u_std, math.log1p(u_n),                 # 13-15
        album_above, artist_above, genre_above,          # 16-18
        alb_gavg, art_gavg, avg_genre_gavg,              # 19-21  (#6)
        math.log1p(alb_nrat), math.log1p(art_nrat),     # 22-23  (#6)
        math.log1p(avg_genre_nrat),                      # 24     (#6)
        cf_score, math.log1p(cf_count),                  # 25-26  (#5)
        mf_score,                                        # 27     (#10)
        v2_score,                                        # 28     (baseline feature)
    ]
```

```python
def compute_cross_features(track_meta, candidates):
    """3 cross-candidate features per candidate in a group of 6 (#7)."""
    metas = [track_meta.get(tid, (None, None, [])) for tid in candidates]
    cross = []
    for i, (alb_i, art_i, genres_i) in enumerate(metas):
        n_same_art = 0
        n_same_alb = 0
        genre_overlap = 0
        gset_i = set(genres_i)
        for j, (alb_j, art_j, genres_j) in enumerate(metas):
            if i == j:
                continue
            if art_i and art_j and art_i == art_j:
                n_same_art += 1
            if alb_i and alb_j and alb_i == alb_j:
                n_same_alb += 1
            if gset_i:
                genre_overlap += len(gset_i & set(genres_j))
        cross.append([float(n_same_art), float(n_same_alb), float(genre_overlap)])
    return cross


# ======================================================================
# 5. Training Data Construction
# ======================================================================

def build_groups(user_ratings, track_meta, user_set, groups_per_user, seed):
    """Build groups of 6 tracks per user, sorted by rating desc (#1).
    Returns [(uid, [6 tids], [6 ratings])] with top-3 first."""
    rng = random.Random(seed)
    track_ids = set(track_meta.keys())
    groups = []

    for uid in user_set:
        ratings = user_ratings[uid]
        user_tracks = [(iid, ratings[iid]) for iid in ratings if iid in track_ids]
        if len(user_tracks) < 6:
            continue
        for _ in range(groups_per_user):
            sample = rng.sample(user_tracks, 6)
            sample.sort(key=lambda x: x[1], reverse=True)
            tids = [t[0] for t in sample]
            rats = [float(t[1]) for t in sample]
            groups.append((uid, tids, rats))
```

```python

    return groups


def featurize_groups(groups, user_ratings, track_meta, global_stats,
                     album_to_tracks, artist_to_tracks,
                     mf_uid_map, mf_iid_map, mf_user_f, mf_item_f):
    """Compute feature matrices for groups.
    Returns X (n_groups, 6, N_FEATURES), ratings (n_groups, 6)."""
    n = len(groups)
    X = np.zeros((n, 6, N_FEATURES), dtype=np.float32)
    R = np.zeros((n, 6), dtype=np.float32)

    for g, (uid, tids, rats) in enumerate(groups):
        ratings = user_ratings[uid]
        u_mean, u_std, u_n = compute_user_profile(ratings)

        for c, tid in enumerate(tids):
            base = compute_base_features(
                ratings, track_meta, tid, u_mean, u_std, u_n,
                global_stats, album_to_tracks, artist_to_tracks,
                mf_uid_map, mf_iid_map, mf_user_f, mf_item_f, uid)
            X[g, c, :N_BASE_FEATURES] = base
            R[g, c] = rats[c]

        cross = compute_cross_features(track_meta, tids)
        for c in range(6):
            X[g, c, N_BASE_FEATURES:] = cross[c]

        if (g + 1) % 50000 == 0:
            print(f"       Featurized {g+1:,}/{n:,} groups ...")

    print(f"       Featurized {n:,} groups -> X{X.shape}")
    return X, R


# ====================================================================
# 6. Feature Normalization (#4)
# ====================================================================

def compute_norm_stats(X):
    """Compute per-feature mean and std from (n_groups, 6, n_feat)."""
    flat = X.reshape(-1, X.shape[-1])
    mean = flat.mean(axis=0)
    std = flat.std(axis=0)
    std[std < 1e-8] = 1.0  # avoid division by zero
    return mean, std
```

```python


def normalize(X, mean, std):
    return (X - mean) / std


# ======================================================================
# 7. Model (#3)
# ======================================================================

class RankingNet(nn.Module):
    def __init__(self, n_features, hidden_sizes=(64, 32), dropout=0.3):
        super().__init__()
        layers = []
        layers.append(nn.BatchNorm1d(n_features))
        prev = n_features
        for h in hidden_sizes:
            layers.append(nn.Linear(prev, h))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout))
            prev = h
        layers.append(nn.Linear(prev, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        # x: (batch, 6, n_features) -> score each candidate
        batch, n_cand, n_feat = x.shape
        flat = x.reshape(batch * n_cand, n_feat)
        scores = self.net(flat)  # (batch*6, 1)
        return scores.reshape(batch, n_cand)


# ======================================================================
# 8. Loss Function (#2, #9)
# ======================================================================

def bpr_group_loss(scores, ratings, hard_neg_temp=20.0):
    """BPR loss over (top-3, bottom-3) pairs within each group.

    scores: (batch, 6) -- model predictions, positions 0-2 are top-3
    ratings: (batch, 6) -- true ratings for hard negative weighting
    """
    pos_scores = scores[:, :3]   # (batch, 3)
    neg_scores = scores[:, 3:]   # (batch, 3)
    pos_ratings = ratings[:, :3]
    neg_ratings = ratings[:, 3:]

```

```
    # All 9 pairs per group: pos_i vs neg_j
    pos_exp = pos_scores.unsqueeze(2)   # (batch, 3, 1)
    neg_exp = neg_scores.unsqueeze(1)   # (batch, 1, 3)
    diff = pos_exp - neg_exp            # (batch, 3, 3)

    # (#9) Hard negative weighting: pairs with close ratings get more
        weight
    pr_exp = pos_ratings.unsqueeze(2)
    nr_exp = neg_ratings.unsqueeze(1)
    rating_gap = (pr_exp - nr_exp).abs()
    weight = torch.exp(-rating_gap / hard_neg_temp)

    pair_loss = -torch.log(torch.sigmoid(diff) + 1e-8) * weight
    return pair_loss.mean()


# ====================================================================
# 9. Training Loop
# ====================================================================

class GroupDataset(Dataset):
    def __init__(self, X, R):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.R = torch.tensor(R, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.R[idx]


def compute_val_auc(model, val_X, val_R):
    """AUC on validation groups: does model rank top-3 above bottom-3?"""
    model.eval()
    dataset = GroupDataset(val_X, val_R)
    loader = DataLoader(dataset, batch_size=4096, shuffle=False)

    correct = 0
    total = 0

    with torch.no_grad():
        for x_batch, r_batch in loader:
            x_batch = x_batch.to(DEVICE)
            scores = model(x_batch).cpu().numpy()  # (batch, 6)

            for i in range(scores.shape[0]):
```

```
                # Positions 0-2 are top-3 by rating, 3-5 are bottom-3
                for p in range(3):
                    for n in range(3, 6):
                        if scores[i, p] > scores[i, n]:
                            correct += 1
                        elif scores[i, p] == scores[i, n]:
                            correct += 0.5
                        total += 1

    return correct / total if total > 0 else 0.0


def train_ranking_model(train_X, train_R, val_X, val_R):
    """Train with BPR loss + early stopping on val AUC."""
    print(f"\n  Training ranking model on {DEVICE} ...")
    print(f"    Train: {train_X.shape[0]:,} groups, "
          f"Val: {val_X.shape[0]:,} groups")

    dataset = GroupDataset(train_X, train_R)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True,
                        num_workers=0,
                        pin_memory=(DEVICE.type == "cuda"))

    model = RankingNet(N_FEATURES, HIDDEN_SIZES, DROPOUT).to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR,
                                 weight_decay=WEIGHT_DECAY)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, MAX_EPOCHS)

    best_auc = 0.0
    best_state = None
    patience_counter = 0

    for epoch in range(MAX_EPOCHS):
        model.train()
        total_loss = 0
        n_batches = 0

        for x_batch, r_batch in loader:
            x_batch = x_batch.to(DEVICE)
            r_batch = r_batch.to(DEVICE)

            scores = model(x_batch)
            loss = bpr_group_loss(scores, r_batch, HARD_NEG_TEMP)

            optimizer.zero_grad()
            loss.backward()
```

```
            optimizer.step()

            total_loss += loss.item()
            n_batches += 1

        scheduler.step()
        avg_loss = total_loss / n_batches

        # Evaluate every 5 epochs or first epoch
        if (epoch + 1) % 5 == 0 or epoch == 0:
            # Training pair accuracy
            model.eval()
            with torch.no_grad():
                sample_x = torch.tensor(
                    train_X[:2048], dtype=torch.float32, device=DEVICE)
                sample_scores = model(sample_x).cpu().numpy()
                train_correct = 0
                train_total = 0
                for i in range(sample_scores.shape[0]):
                    for p in range(3):
                        for n in range(3, 6):
                            if sample_scores[i, p] > sample_scores[i, n]:
                                train_correct += 1
                            elif sample_scores[i, p] == sample_scores[i, n
                                ]:
                                train_correct += 0.5
                            train_total += 1
                train_acc = train_correct / train_total

            val_auc = compute_val_auc(model, val_X, val_R)
            improved = ""

            if val_auc > best_auc:
                best_auc = val_auc
                best_state = {k: v.cpu().clone()
                              for k, v in model.state_dict().items()}
                patience_counter = 0
                improved = " *BEST*"
            else:
                patience_counter += 5

            print(f"    Epoch {epoch+1:3d}/{MAX_EPOCHS}  "
                  f"loss={avg_loss:.4f}  "
                  f"train_acc={train_acc:.4f}  "
                  f"val_AUC={val_auc:.4f}{improved}")

            if patience_counter >= PATIENCE:
```

```
                print(f"    Early stopping at epoch {epoch+1}")
                break

    # Load best model
    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(DEVICE)
    print(f"    Best val AUC: {best_auc:.4f}")
    return model, best_auc


# ======================================================================
# 10. Prediction & Submission
# ======================================================================

def predict_test(model, user_ratings, track_meta, test_users,
                 feat_mean, feat_std,
                 global_stats, album_to_tracks, artist_to_tracks,
                 mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
                 out_path, model_name="model"):
    """Score test candidates and generate submission."""
    print(f"\n>>> Predicting with {model_name} ...")
    model.eval()
    results = []

    with torch.no_grad():
        for uid, candidates in test_users:
            ratings = user_ratings.get(uid, {})
            u_mean, u_std, u_n = compute_user_profile(ratings)

            # Base features
            base_feats = []
            for tid in candidates:
                bf = compute_base_features(
                    ratings, track_meta, tid, u_mean, u_std, u_n,
                    global_stats, album_to_tracks, artist_to_tracks,
                    mf_uid_map, mf_iid_map, mf_user_f, mf_item_f, uid)
                base_feats.append(bf)

            # Cross features
            cross = compute_cross_features(track_meta, candidates)

            # Combine
            X = np.zeros((1, 6, N_FEATURES), dtype=np.float32)
            for c in range(6):
                X[0, c, :N_BASE_FEATURES] = base_feats[c]
                X[0, c, N_BASE_FEATURES:] = cross[c]
```

```python

            # Normalize
            X = normalize(X, feat_mean, feat_std)

            X_t = torch.tensor(X, dtype=torch.float32, device=DEVICE)
            scores = model(X_t).cpu().numpy()[0]  # (6,)

            ranked = sorted(zip(scores, candidates), reverse=True)
            top3 = set(tid for _, tid in ranked[:3])
            for tid in candidates:
                results.append((f"{uid}_{tid}", 1 if tid in top3 else 0))

    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")

    ones = sum(v for _, v in results)
    print(f"    {len(results):,} predictions ({ones:,} rec, "
          f"{len(results)-ones:,} don't)")
    print(f"    Saved -> {out_path}")
    return results


# ======================================================================
# 11. V2 Heuristic Baseline (for comparison)
# ======================================================================

def v2_heuristic_val_auc(user_ratings, track_meta, val_groups):
    """Evaluate v2 heuristic on the same validation groups."""
    correct = 0
    total = 0
    for uid, tids, rats in val_groups:
        ratings = user_ratings[uid]
        scores = []
        for tid in tids:
            alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            scores.append(s)
```

```python

        for p in range(3):
            for n in range(3, 6):
                if scores[p] > scores[n]:
                    correct += 1
                elif scores[p] == scores[n]:
                    correct += 0.5
                total += 1

    return correct / total if total > 0 else 0.0


# ====================================================================
# 12. Main
# ====================================================================

def main():
    tee = Tee(RESULTS_FILE)
    sys.stdout = tee
    t0 = time.time()

    torch.manual_seed(SEED)
    random.seed(SEED)
    np.random.seed(SEED)

    print("=" * 60)
    print("  EE627 Midterm v5 - Full Model")
    print("=" * 60)
    print(f"  Device: {DEVICE}")
    if torch.cuda.is_available():
        print(f"  GPU: {torch.cuda.get_device_name(0)}")
    print()

    # ---- Load data ----
    user_ratings = parse_training(os.path.join(DATA_DIR, "trainItem2.txt")
        )
    track_meta = parse_tracks(os.path.join(DATA_DIR, "trackData2.txt"))
    album_meta = parse_albums(os.path.join(DATA_DIR, "albumData2.txt"))
    test_users = parse_test(os.path.join(DATA_DIR, "testItem2.txt"))

    # ---- Enrich ----
    print("\n" + "=" * 60)
    print("  ENRICHING TRACKS")
    print("=" * 60)
    enriched_meta = enrich_tracks_with_albums(track_meta, album_meta)

    # ---- Auxiliary structures ----
```

```
    print("\n" + "=" * 60)
    print("  BUILDING AUXILIARY STRUCTURES")
    print("=" * 60)
    global_stats = build_global_item_stats(user_ratings)
    album_to_tracks, artist_to_tracks = build_track_mappings(enriched_meta
        )

    # (#10) Matrix factorization
    mf_uid_map, mf_iid_map, mf_user_f, mf_item_f = train_mf_embeddings(
        user_ratings, MF_FACTORS, MF_EPOCHS, MF_LR, MF_BATCH)

    # ---- Split users for train/val ----
    print("\n" + "=" * 60)
    print("  BUILDING TRAINING & VALIDATION DATA")
    print("=" * 60)

    rng = random.Random(SEED)
    all_uids = list(user_ratings.keys())
    rng.shuffle(all_uids)
    split = int(len(all_uids) * 0.9)
    train_uids = set(all_uids[:split])
    val_uids = set(all_uids[split:])
    print(f"  Train users: {len(train_uids):,}, Val users: {len(val_uids)
        :,}")

    # (#1) Build groups of 6
    t1 = time.time()
    train_groups = build_groups(user_ratings, enriched_meta, train_uids,
                                GROUPS_PER_USER, SEED)
    val_groups = build_groups(user_ratings, enriched_meta, val_uids,
                              10, SEED + 1)
    print(f"  Train groups: {len(train_groups):,}, "
          f"Val groups: {len(val_groups):,}")

    # Featurize
    print("\n  Featurizing training groups ...")
    train_X, train_R = featurize_groups(
        train_groups, user_ratings, enriched_meta, global_stats,
        album_to_tracks, artist_to_tracks,
        mf_uid_map, mf_iid_map, mf_user_f, mf_item_f)

    print("  Featurizing validation groups ...")
    val_X, val_R = featurize_groups(
        val_groups, user_ratings, enriched_meta, global_stats,
        album_to_tracks, artist_to_tracks,
        mf_uid_map, mf_iid_map, mf_user_f, mf_item_f)

```

```
    t2 = time.time()
    print(f"  Feature computation: {t2-t1:.1f}s")

    # (#4) Normalize
    feat_mean, feat_std = compute_norm_stats(train_X)
    train_X = normalize(train_X, feat_mean, feat_std)
    val_X = normalize(val_X, feat_mean, feat_std)

    # ---- V2 baseline on same val groups ----
    print("\n" + "=" * 60)
    print("  V2 HEURISTIC BASELINE")
    print("=" * 60)
    v2_auc = v2_heuristic_val_auc(user_ratings, enriched_meta, val_groups)
    print(f"  v2 heuristic val AUC: {v2_auc:.4f}  (Kaggle: 0.871)")

    # ---- Train ----
    print("\n" + "=" * 60)
    print("  TRAINING RANKING MODEL")
    print("=" * 60)
    model, best_auc = train_ranking_model(train_X, train_R, val_X, val_R)

    # ---- Generate submissions ----
    print("\n" + "=" * 60)
    print("  GENERATING SUBMISSIONS")
    print("=" * 60)

    v5_results = predict_test(
        model, user_ratings, enriched_meta, test_users,
        feat_mean, feat_std,
        global_stats, album_to_tracks, artist_to_tracks,
        mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
        os.path.join(OUTPUT_DIR, "submission_v5b_nn.csv"),
        "v5 Ranking NN")

    # ---- Generate v2 baseline predictions + confidence-gated hybrid ----
    print("\n" + "=" * 60)
    print("  V2 vs V5 AGREEMENT & HYBRID SUBMISSION")
    print("=" * 60)

    v2_results = []
    hybrid_results = []  # use v2 when confident, v5 when v2 is uncertain

    # Build v5 results lookup by user for hybrid
    v5_by_user = {}  # uid -> {tid: label}
    idx = 0
    for uid, candidates in test_users:
        v5_by_user[uid] = {}
```

```
        for tid in candidates:
            v5_by_user[uid][tid] = v5_results[idx][1]
            idx += 1

    for uid, candidates in test_users:
        ratings = user_ratings.get(uid, {})

        # Compute v2 scores for this user's 6 candidates
        v2_scores = []
        for tid in candidates:
            alb_id, art_id, genre_ids = enriched_meta.get(
                tid, (None, None, []))
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            v2_scores.append(s)

        # v2 predictions
        ranked = sorted(zip(v2_scores, candidates), reverse=True)
        v2_top3 = set(tid for _, tid in ranked[:3])
        for tid in candidates:
            v2_results.append((f"{uid}_{tid}", 1 if tid in v2_top3 else 0)
                )

        # Confidence = gap between 3rd and 4th ranked candidate
        sorted_scores = sorted(v2_scores, reverse=True)
        confidence_gap = sorted_scores[2] - sorted_scores[3]

        # If v2 is confident (clear gap), keep v2. Otherwise use v5.
        # Threshold tuned: gap=0 means all tied, gap>10 means clear winner
        if confidence_gap > 5.0:
            # v2 is confident -- trust it
            for tid in candidates:
                hybrid_results.append(
                    (f"{uid}_{tid}", 1 if tid in v2_top3 else 0))
        else:
            # v2 is uncertain -- let v5 model decide
            for tid in candidates:
                hybrid_results.append(
                    (f"{uid}_{tid}", v5_by_user[uid][tid]))

    # Write hybrid submission
```

```
    hybrid_path = os.path.join(OUTPUT_DIR, "submission_v5b_hybrid.csv")
    with open(hybrid_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in hybrid_results:
            f.write(f"{key},{label}\n")
    print(f"  Hybrid submission saved -> {hybrid_path}")

    # Count how many users fall into each bucket
    n_v2_confident = 0
    n_v5_override = 0
    for uid, candidates in test_users:
        v2_scores_quick = []
        ratings = user_ratings.get(uid, {})
        for tid in candidates:
            alb_id, art_id, genre_ids = enriched_meta.get(
                tid, (None, None, []))
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            v2_scores_quick.append(s)
        ss = sorted(v2_scores_quick, reverse=True)
        if ss[2] - ss[3] > 5.0:
            n_v2_confident += 1
        else:
            n_v5_override += 1

    print(f"\n  Hybrid breakdown:")
    print(f"    v2 confident (gap>5):  {n_v2_confident:,} users "
          f"({100*n_v2_confident/len(test_users):.1f}%) -- kept v2")
    print(f"    v2 uncertain (gap<=5): {n_v5_override:,} users "
          f"({100*n_v5_override/len(test_users):.1f}%) -- used v5 model")

    # Compare v5 vs v2
    agree = sum(1 for (_, a), (_, b) in zip(v5_results, v2_results) if a
        == b)
    total_preds = len(v5_results)
    agree_pct = 100 * agree / total_preds

    v5_promotes = sum(1 for (_, v5), (_, v2) in zip(v5_results, v2_results
        )
                      if v5 == 1 and v2 == 0)
    v5_demotes = sum(1 for (_, v5), (_, v2) in zip(v5_results, v2_results)
```

```
                    if v5 == 0 and v2 == 1)

    # Compare hybrid vs v2
    h_agree = sum(1 for (_, a), (_, b) in zip(hybrid_results, v2_results)
                  if a == b)
    h_agree_pct = 100 * h_agree / total_preds

    print(f"\n  v5 vs v2:")
    print(f"    Agreement:   {agree:,}/{total_preds:,} ({agree_pct:.1f}%)"
        )
    print(f"    Promotes:    {v5_promotes:,}  Demotes: {v5_demotes:,}")

    print(f"\n  hybrid vs v2:")
    print(f"    Agreement:   {h_agree:,}/{total_preds:,} ({h_agree_pct:.1f
        }%)")

    print()
    if agree_pct >= 95:
        print("  VERDICT v5: Very close to v2 -- safe to submit")
    elif agree_pct >= 90:
        print("  VERDICT v5: Moderate divergence -- could go either way")
    elif agree_pct >= 80:
        print("  VERDICT v5: Significant divergence -- risky")
    else:
        print("  VERDICT v5: Major divergence -- likely worse than v2")

    print(f"  VERDICT hybrid: {h_agree_pct:.1f}% agreement -- "
          f"safest bet, changes only uncertain users")

    # ---- Summary ----
    elapsed = time.time() - t0
    print("\n" + "=" * 60)
    print("  FINAL RESULTS")
    print("=" * 60)
    print(f"  v2 heuristic val AUC:  {v2_auc:.4f}  (Kaggle: 0.871)")
    print(f"  v5 ranking NN val AUC: {best_auc:.4f}")
    print(f"  Delta:                 {best_auc - v2_auc:+.4f}")
    print(f"  Agreement with v2:     {agree_pct:.1f}%")
    print()
    print(f"  Features: {N_FEATURES}")
    print(f"  Training groups: {len(train_groups):,}")
    print(f"  Validation groups: {len(val_groups):,}")
    print(f"  MF factors: {MF_FACTORS}")
    print()
    print(f"  v5 agreement with v2:  {agree_pct:.1f}%")
    print(f"  hybrid agreement:      {h_agree_pct:.1f}%")
    print()
```

```
    print(f"  Submissions:")
    print(f"    submission_v5b_nn.csv      -- pure model")
    print(f"    submission_v5b_hybrid.csv  -- v2 when confident, v5 when
        uncertain")
    print(f"  Total time: {elapsed:.1f}s ({elapsed/60:.1f}m)")
    print("\nDone.")

    tee.close()


if __name__ == "__main__":
    main()
```

Listing 3: EE627_Midterm_v5_full.py (v5b hyperparameters)

A.4 v6: Refined Model with Dual MF and Ensemble

```python
"""
EE627A Midterm v6 - Refined Model
Jude Eschete

Building on v5's 0.878 score. Key changes inspired by HW6 (feature selection):

  1. Bigger MF (k=128) + longer training (80 epochs) for richer embeddings
  2. Feature selection: train once, rank features by importance, retrain
     with only the top features (HW6 lesson: fewer features = less overfit)
  3. Multiple MF ranks (k=32 + k=128) as separate features -- different
     factorization depths capture different patterns
  5. Neighbor-based CF: find similar users via MF cosine similarity,
     average their ratings for candidate's album/artist as new features
  7. Ensemble 3 models with different seeds, average scores before ranking

Uses RTX 5080 GPU.
"""

import os
import sys
import random
import math
import time
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, TensorDataset
from collections import defaultdict

# ======================================================================
# Configuration
# ======================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "Data", "music-recommender-2026s")

OUTPUT_DIR = SCRIPT_DIR
RESULTS_FILE = os.path.join(SCRIPT_DIR, "results.txt")
SEED = 42
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# MF Hyperparameters (#1, #3)
MF_CONFIGS = [
    {"factors": 32,  "epochs": 40, "lr": 0.01,  "label": "mf32"},
```

```
    {"factors": 128, "epochs": 80, "lr": 0.003, "label": "mf128"},
]
MF_BATCH = 65536

# Training
GROUPS_PER_USER = 30
HIDDEN_SIZES = (64, 32)   # back to simpler net (HW6 lesson)
DROPOUT = 0.3
LR = 1e-3
WEIGHT_DECAY = 1e-4
BATCH_SIZE = 2048
MAX_EPOCHS = 100
PATIENCE = 15
HARD_NEG_TEMP = 15.0

# Neighbor CF (#5)
N_NEIGHBORS = 20

# Ensemble (#7)
N_ENSEMBLE = 3


# ======================================================================
# Logging
# ======================================================================

class Tee:
    def __init__(self, filepath):
        self.file = open(filepath, "w", encoding="utf-8")
        self.stdout = sys.stdout

    def write(self, data):
        self.stdout.write(data)
        self.file.write(data)

    def flush(self):
        self.stdout.flush()
        self.file.flush()

    def close(self):
        self.file.close()
        sys.stdout = self.stdout


# ======================================================================
# 1. Data Parsing
# ======================================================================
```

```python

def parse_training(path):
    print("[1/4] Loading training data ...")
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"       {len(users):,} users, {total:,} ratings")
    return users


def parse_tracks(path):
    print("[2/4] Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"       {len(tracks):,} tracks")
    return tracks


def parse_albums(path):
    print("[3/4] Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
```

```
            art = None if len(parts) < 2 or parts[1] == "None" else int(parts[1])
            genres = []
            for g in parts[2:]:
                g = g.strip()
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    print(f"       {len(albums):,} albums")
    return albums


def parse_test(path):
    print("[4/4] Loading test data ...")
    test = []
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    print(f"       {len(test):,} test users, "
          f"{sum(len(t) for _, t in test):,} candidates")
    return test


# ======================================================================
# 2. Album Enrichment
# ======================================================================

def enrich_tracks_with_albums(tracks, albums):
    enriched = {}
    stats = {"artist_filled": 0, "genres_added": 0}
    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
```

```
                new_art = album_art
                stats["artist_filled"] += 1
            track_genre_set = set(genres)
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    stats["genres_added"] += 1
        enriched[tid] = (alb, new_art, new_genres)
    print(f"  Enrichment: {stats['artist_filled']:,} artists filled, "
          f"{stats['genres_added']:,} genres added")
    return enriched


# ======================================================================
# 3. Auxiliary Data Structures
# ======================================================================

def build_global_item_stats(user_ratings):
    print("  Building global item stats ...")
    item_sum = defaultdict(float)
    item_cnt = defaultdict(int)
    for ratings in user_ratings.values():
        for iid, r in ratings.items():
            item_sum[iid] += r
            item_cnt[iid] += 1
    stats = {iid: (item_sum[iid] / item_cnt[iid], item_cnt[iid])
             for iid in item_sum}
    print(f"       {len(stats):,} items with global stats")
    return stats


def build_track_mappings(track_meta):
    print("  Building track mappings ...")
    album_to_tracks = defaultdict(list)
    artist_to_tracks = defaultdict(list)
    for tid, (alb, art, genres) in track_meta.items():
        if alb is not None:
            album_to_tracks[alb].append(tid)
        if art is not None:
            artist_to_tracks[art].append(tid)
    print(f"       {len(album_to_tracks):,} albums, "
          f"{len(artist_to_tracks):,} artists mapped")
    return dict(album_to_tracks), dict(artist_to_tracks)


# ======================================================================
```

```python
# 4. Multiple MF Models (#1, #3)
# ======================================================================

def train_mf(user_ratings, n_factors, n_epochs, lr, batch_size, label):
    """Train one MF model. Returns (uid_map, iid_map, user_factors, item_factors)."""
    print(f"\n  Training MF [{label}] (k={n_factors}, {n_epochs} epochs) ...")

    uid_to_idx = {}
    iid_to_idx = {}
    u_list, i_list, r_list = [], [], []

    for uid, ratings in user_ratings.items():
        if uid not in uid_to_idx:
            uid_to_idx[uid] = len(uid_to_idx)
        u_idx = uid_to_idx[uid]
        for iid, r in ratings.items():
            if iid not in iid_to_idx:
                iid_to_idx[iid] = len(iid_to_idx)
            u_list.append(u_idx)
            i_list.append(iid_to_idx[iid])
            r_list.append(float(r))

    n_users = len(uid_to_idx)
    n_items = len(iid_to_idx)

    users_t = torch.tensor(u_list, dtype=torch.long)
    items_t = torch.tensor(i_list, dtype=torch.long)
    ratings_t = torch.tensor(r_list, dtype=torch.float32)

    dataset = TensorDataset(users_t, items_t, ratings_t)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
                        num_workers=0, pin_memory=(DEVICE.type == "cuda"))

    user_emb = nn.Embedding(n_users, n_factors).to(DEVICE)
    item_emb = nn.Embedding(n_items, n_factors).to(DEVICE)
    nn.init.normal_(user_emb.weight, 0, 0.1)
    nn.init.normal_(item_emb.weight, 0, 0.1)

    optimizer = torch.optim.Adam(
        list(user_emb.parameters()) + list(item_emb.parameters()),
        lr=lr, weight_decay=1e-4)

    for epoch in range(n_epochs):
        total_loss = 0
        n_b = 0
```

```
        for u_b, i_b, r_b in loader:
            u_b, i_b, r_b = u_b.to(DEVICE), i_b.to(DEVICE), r_b.to(DEVICE)
            pred = (user_emb(u_b) * item_emb(i_b)).sum(dim=1)
            loss = ((pred - r_b) ** 2).mean()
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            n_b += 1
        if (epoch + 1) % 10 == 0 or epoch == 0:
            print(f"    [{label}] Epoch {epoch+1:3d}/{n_epochs}  "
                  f"MSE={total_loss/n_b:.2f}")

    uf = user_emb.weight.detach().cpu().numpy()
    itf = item_emb.weight.detach().cpu().numpy()
    print(f"    [{label}] Done. User: {uf.shape}, Item: {itf.shape}")
    return uid_to_idx, iid_to_idx, uf, itf


# ======================================================================
# 5. Neighbor CF (#5)
# ======================================================================

def build_neighbor_index(mf_uid_map, mf_user_f, user_ratings, n_neighbors
    =20):
    """For each user, find N most similar users by MF cosine similarity.
    Returns {uid: [(neighbor_uid, similarity), ...]}."""
    print(f"\n  Building neighbor index (k={n_neighbors}) ...")
    t0 = time.time()

    # Normalize user factors for cosine similarity
    norms = np.linalg.norm(mf_user_f, axis=1, keepdims=True)
    norms[norms < 1e-8] = 1.0
    normed = mf_user_f / norms

    # Reverse map: idx -> uid
    idx_to_uid = {v: k for k, v in mf_uid_map.items()}

    # Only compute for users who appear in test or train
    uid_list = list(mf_uid_map.keys())
    neighbors = {}

    # Process in batches to avoid OOM
    batch_size = 1000
    for start in range(0, len(uid_list), batch_size):
        batch_uids = uid_list[start:start + batch_size]
        batch_idxs = [mf_uid_map[u] for u in batch_uids]
```

```
        batch_vecs = normed[batch_idxs]  # (batch, k)

        # Cosine sim against all users
        sims = batch_vecs @ normed.T  # (batch, n_users)

        for i, uid in enumerate(batch_uids):
            my_idx = batch_idxs[i]
            sim_row = sims[i]
            sim_row[my_idx] = -1.0  # exclude self

            top_idxs = np.argpartition(sim_row, -n_neighbors)[-n_neighbors
                :]
            top_idxs = top_idxs[np.argsort(sim_row[top_idxs])[::-1]]

            nbrs = []
            for ni in top_idxs:
                if ni in idx_to_uid:
                    nbrs.append((idx_to_uid[ni], float(sim_row[ni])))
            neighbors[uid] = nbrs

    t1 = time.time()
    print(f"       Built neighbors for {len(neighbors):,} users "
          f"in {t1-t0:.1f}s")
    return neighbors


def compute_neighbor_features(uid, tid, track_meta, user_ratings,
                              neighbor_index):
    """CF features from similar users' ratings (#5).
    Returns [nbr_album_score, nbr_artist_score, nbr_coverage]."""
    alb_id, art_id, _ = track_meta.get(tid, (None, None, []))
    neighbors = neighbor_index.get(uid, [])

    alb_scores = []
    art_scores = []

    for nbr_uid, sim in neighbors:
        nbr_ratings = user_ratings.get(nbr_uid, {})
        if alb_id is not None and alb_id in nbr_ratings:
            alb_scores.append(nbr_ratings[alb_id] * sim)
        if art_id is not None and art_id in nbr_ratings:
            art_scores.append(nbr_ratings[art_id] * sim)

    nbr_album = sum(alb_scores) / len(alb_scores) if alb_scores else 0.0
    nbr_artist = sum(art_scores) / len(art_scores) if art_scores else 0.0
    nbr_coverage = float(len(alb_scores) + len(art_scores))

```

```
    return [nbr_album, nbr_artist, nbr_coverage]


# ======================================================================
# 6. Feature Engineering
# ======================================================================

def compute_user_profile(ratings):
    vals = list(ratings.values())
    n = len(vals)
    if n == 0:
        return 0.0, 0.0, 0
    mean = sum(vals) / n
    var = sum((v - mean) ** 2 for v in vals) / n if n > 1 else 0.0
    return mean, var ** 0.5, n


def compute_base_features(ratings, track_meta, tid, u_mean, u_std, u_n,
                          global_stats, album_to_tracks, artist_to_tracks,
                          mf_models, uid, neighbor_index, user_ratings):
    """Base features per (user, track) pair.
    mf_models is a list of (uid_map, iid_map, user_f, item_f)."""
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    # --- Album ---
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = float(ratings[alb_id]), 1.0
    else:
        album_score, has_album = 0.0, 0.0

    # --- Artist ---
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = float(ratings[art_id]), 1.0
    else:
        artist_score, has_artist = 0.0, 0.0

    # --- Genre stats ---
    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    gc = len(genre_scores)
    if gc > 0:
        g_max = float(max(genre_scores))
        g_min = float(min(genre_scores))
        g_mean = sum(genre_scores) / gc
        g_var = sum((s - g_mean) ** 2 for s in genre_scores) / gc
    else:
        g_max = g_min = g_mean = g_var = 0.0
```

```python

    evidence_count = has_album + has_artist + (1.0 if gc > 0 else 0.0)

    # --- Interactions ---
    aa_int = album_score * artist_score / 100.0 if (has_album and has_artist) else 0.0
    ga_int = g_mean * artist_score / 100.0 if (has_artist and gc > 0) else 0.0
    ag_int = album_score * g_mean / 100.0 if (has_album and gc > 0) else 0.0

    # --- User-relative ---
    album_above = (album_score - u_mean) if has_album else 0.0
    artist_above = (artist_score - u_mean) if has_artist else 0.0
    genre_above = (g_mean - u_mean) if gc > 0 else 0.0

    # --- Popularity & global avg ---
    alb_gavg, alb_nrat = global_stats.get(alb_id, (0.0, 0)) if alb_id else (0.0, 0)
    art_gavg, art_nrat = global_stats.get(art_id, (0.0, 0)) if art_id else (0.0, 0)
    g_gavg_sum, g_nrat_sum, g_items = 0.0, 0.0, 0
    for gid in genre_ids:
        if gid in global_stats:
            gavg, nrat = global_stats[gid]
            g_gavg_sum += gavg
            g_nrat_sum += nrat
            g_items += 1
    avg_genre_gavg = g_gavg_sum / g_items if g_items > 0 else 0.0
    avg_genre_nrat = g_nrat_sum / g_items if g_items > 0 else 0.0

    # --- CF sibling features ---
    sibling_ratings = []
    if alb_id is not None:
        for sib in album_to_tracks.get(alb_id, []):
            if sib != tid and sib in ratings:
                sibling_ratings.append(ratings[sib])
    if art_id is not None:
        album_sibs = set(album_to_tracks.get(alb_id, [])) if alb_id else set()
        for sib in artist_to_tracks.get(art_id, []):
            if sib != tid and sib not in album_sibs and sib in ratings:
                sibling_ratings.append(ratings[sib])
    cf_score = sum(sibling_ratings) / len(sibling_ratings) if sibling_ratings else 0.0
    cf_count = len(sibling_ratings)

```

```
    # --- MF dot products (#3: multiple ranks) ---
    mf_scores = []
    for mf_uid_map, mf_iid_map, mf_user_f, mf_item_f in mf_models:
        if uid in mf_uid_map and tid in mf_iid_map:
            mf_scores.append(float(
                mf_user_f[mf_uid_map[uid]] @ mf_item_f[mf_iid_map[tid]]))
        else:
            mf_scores.append(0.0)

    # --- Neighbor CF (#5) ---
    nbr_feats = compute_neighbor_features(
        uid, tid, track_meta, user_ratings, neighbor_index)

    # --- v2 heuristic score ---
    v2_score = (has_album * album_score + has_artist * artist_score
                + gc * g_mean / 10.0)

    feats = [
        album_score, has_album,                          # 0-1
        artist_score, has_artist,                        # 2-3
        float(gc), g_max, g_min, g_mean, g_var,         # 4-8
        evidence_count,                                  # 9
        aa_int, ga_int, ag_int,                          # 10-12
        u_mean, u_std, math.log1p(u_n),                 # 13-15
        album_above, artist_above, genre_above,          # 16-18
        alb_gavg, art_gavg, avg_genre_gavg,              # 19-21
        math.log1p(alb_nrat), math.log1p(art_nrat),     # 22-23
        math.log1p(avg_genre_nrat),                      # 24
        cf_score, math.log1p(cf_count),                  # 25-26
        v2_score,                                        # 27
    ]
    feats.extend(mf_scores)        # 28-29 (two MF scores)
    feats.extend(nbr_feats)        # 30-32 (neighbor CF)

    return feats


def compute_cross_features(track_meta, candidates):
    """3 cross-candidate features per candidate in a group of 6."""
    metas = [track_meta.get(tid, (None, None, [])) for tid in candidates]
    cross = []
    for i, (alb_i, art_i, genres_i) in enumerate(metas):
        n_same_art = 0
        n_same_alb = 0
        genre_overlap = 0
        gset_i = set(genres_i)
        for j, (alb_j, art_j, genres_j) in enumerate(metas):
```

```
            if i == j:
                continue
            if art_i and art_j and art_i == art_j:
                n_same_art += 1
            if alb_i and alb_j and alb_i == alb_j:
                n_same_alb += 1
            if gset_i:
                genre_overlap += len(gset_i & set(genres_j))
        cross.append([float(n_same_art), float(n_same_alb), float(genre_overlap)])
    return cross


# Total features: 33 base + 3 cross = 36
N_BASE_FEATURES = 33
N_CROSS_FEATURES = 3
N_FEATURES = N_BASE_FEATURES + N_CROSS_FEATURES


# ======================================================================
# 7. Training Data
# ======================================================================

def build_groups(user_ratings, track_meta, user_set, groups_per_user, seed):
    rng = random.Random(seed)
    track_ids = set(track_meta.keys())
    groups = []
    for uid in user_set:
        ratings = user_ratings[uid]
        user_tracks = [(iid, ratings[iid]) for iid in ratings if iid in track_ids]
        if len(user_tracks) < 6:
            continue
        for _ in range(groups_per_user):
            sample = rng.sample(user_tracks, 6)
            sample.sort(key=lambda x: x[1], reverse=True)
            tids = [t[0] for t in sample]
            rats = [float(t[1]) for t in sample]
            groups.append((uid, tids, rats))
    return groups


def featurize_groups(groups, user_ratings, track_meta, global_stats,
                     album_to_tracks, artist_to_tracks,
                     mf_models, neighbor_index):
    n = len(groups)
```

```
    X = np.zeros((n, 6, N_FEATURES), dtype=np.float32)
    R = np.zeros((n, 6), dtype=np.float32)

    for g, (uid, tids, rats) in enumerate(groups):
        ratings = user_ratings[uid]
        u_mean, u_std, u_n = compute_user_profile(ratings)

        for c, tid in enumerate(tids):
            base = compute_base_features(
                ratings, track_meta, tid, u_mean, u_std, u_n,
                global_stats, album_to_tracks, artist_to_tracks,
                mf_models, uid, neighbor_index, user_ratings)
            X[g, c, :N_BASE_FEATURES] = base
            R[g, c] = rats[c]

        cross = compute_cross_features(track_meta, tids)
        for c in range(6):
            X[g, c, N_BASE_FEATURES:] = cross[c]

        if (g + 1) % 50000 == 0:
            print(f"       Featurized {g+1:,}/{n:,} groups ...")

    print(f"       Featurized {n:,} groups -> X{X.shape}")
    return X, R


# ======================================================================
# 8. Feature Normalization
# ======================================================================

def compute_norm_stats(X):
    flat = X.reshape(-1, X.shape[-1])
    mean = flat.mean(axis=0)
    std = flat.std(axis=0)
    std[std < 1e-8] = 1.0
    return mean, std


def normalize(X, mean, std):
    return (X - mean) / std


# ======================================================================
# 9. Feature Selection (#2)
# ======================================================================

def feature_importance(model, val_X, val_R):
```

```
    """Permutation importance: for each feature, shuffle it and
    measure AUC drop. Bigger drop = more important."""
    print("\n  Computing feature importance (permutation) ...")
    base_auc = compute_val_auc(model, val_X, val_R)
    print(f"    Base val AUC: {base_auc:.4f}")

    n_features = val_X.shape[2]
    importances = []

    for f in range(n_features):
        # Shuffle feature f across all candidates
        X_perm = val_X.copy()
        flat = X_perm.reshape(-1, n_features)
        rng = np.random.RandomState(SEED + f)
        rng.shuffle(flat[:, f])
        X_perm = flat.reshape(val_X.shape)

        perm_auc = compute_val_auc(model, X_perm, val_R)
        drop = base_auc - perm_auc
        importances.append((f, drop))

    importances.sort(key=lambda x: x[1], reverse=True)

    print("    Feature importance (top 15):")
    for rank, (f_idx, drop) in enumerate(importances[:15]):
        print(f"      #{rank+1:2d}  feature[{f_idx:2d}]  "
              f"AUC drop = {drop:+.4f}")

    print(f"    Bottom 5:")
    for f_idx, drop in importances[-5:]:
        print(f"          feature[{f_idx:2d}]  "
              f"AUC drop = {drop:+.4f}")

    return importances


def select_features(X, importances, min_drop=0.0001):
    """Keep only features with importance above threshold."""
    selected = [f_idx for f_idx, drop in importances if drop > min_drop]
    if len(selected) < 5:
        # Keep at least top 10
        selected = [f_idx for f_idx, _ in importances[:10]]
    selected.sort()
    return selected


def apply_feature_mask(X, feature_indices):
```

```
    """Reduce feature dimension to selected indices."""
    return X[:, :, feature_indices]


# ======================================================================
# 10. Model
# ======================================================================

class RankingNet(nn.Module):
    def __init__(self, n_features, hidden_sizes=(64, 32), dropout=0.3):
        super().__init__()
        layers = []
        layers.append(nn.BatchNorm1d(n_features))
        prev = n_features
        for h in hidden_sizes:
            layers.append(nn.Linear(prev, h))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout))
            prev = h
        layers.append(nn.Linear(prev, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        batch, n_cand, n_feat = x.shape
        flat = x.reshape(batch * n_cand, n_feat)
        scores = self.net(flat)
        return scores.reshape(batch, n_cand)


# ======================================================================
# 11. Loss
# ======================================================================

def bpr_group_loss(scores, ratings, hard_neg_temp=15.0):
    pos_scores = scores[:, :3]
    neg_scores = scores[:, 3:]
    pos_ratings = ratings[:, :3]
    neg_ratings = ratings[:, 3:]

    pos_exp = pos_scores.unsqueeze(2)
    neg_exp = neg_scores.unsqueeze(1)
    diff = pos_exp - neg_exp

    pr_exp = pos_ratings.unsqueeze(2)
    nr_exp = neg_ratings.unsqueeze(1)
    rating_gap = (pr_exp - nr_exp).abs()
    weight = torch.exp(-rating_gap / hard_neg_temp)
```

```python

    pair_loss = -torch.log(torch.sigmoid(diff) + 1e-8) * weight
    return pair_loss.mean()


# ======================================================================
# 12. Training
# ======================================================================

class GroupDataset(Dataset):
    def __init__(self, X, R):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.R = torch.tensor(R, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.R[idx]


def compute_val_auc(model, val_X, val_R):
    model.eval()
    dataset = GroupDataset(val_X, val_R)
    loader = DataLoader(dataset, batch_size=4096, shuffle=False)
    correct = 0
    total = 0
    with torch.no_grad():
        for x_batch, r_batch in loader:
            x_batch = x_batch.to(DEVICE)
            scores = model(x_batch).cpu().numpy()
            for i in range(scores.shape[0]):
                for p in range(3):
                    for n in range(3, 6):
                        if scores[i, p] > scores[i, n]:
                            correct += 1
                        elif scores[i, p] == scores[i, n]:
                            correct += 0.5
                        total += 1
    return correct / total if total > 0 else 0.0


def train_ranking_model(train_X, train_R, val_X, val_R, n_features,
                        seed_offset=0):
    """Train one model. seed_offset allows different seeds for ensemble."""
    torch.manual_seed(SEED + seed_offset)
```

```
    np.random.seed(SEED + seed_offset)

    dataset = GroupDataset(train_X, train_R)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True,
                        num_workers=0,
                        pin_memory=(DEVICE.type == "cuda"))

    model = RankingNet(n_features, HIDDEN_SIZES, DROPOUT).to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR,
                                 weight_decay=WEIGHT_DECAY)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, MAX_EPOCHS)

    best_auc = 0.0
    best_state = None
    patience_counter = 0

    for epoch in range(MAX_EPOCHS):
        model.train()
        total_loss = 0
        n_batches = 0

        for x_batch, r_batch in loader:
            x_batch = x_batch.to(DEVICE)
            r_batch = r_batch.to(DEVICE)
            scores = model(x_batch)
            loss = bpr_group_loss(scores, r_batch, HARD_NEG_TEMP)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            n_batches += 1

        scheduler.step()
        avg_loss = total_loss / n_batches

        if (epoch + 1) % 5 == 0 or epoch == 0:
            model.eval()
            with torch.no_grad():
                sample_x = torch.tensor(
                    train_X[:2048], dtype=torch.float32, device=DEVICE)
                sample_scores = model(sample_x).cpu().numpy()
                train_correct = 0
                train_total = 0
                for i in range(sample_scores.shape[0]):
                    for p in range(3):
                        for n in range(3, 6):
```

```
                            if sample_scores[i, p] > sample_scores[i, n]:
                                train_correct += 1
                            elif sample_scores[i, p] == sample_scores[i, n]:
                                train_correct += 0.5
                            train_total += 1
                train_acc = train_correct / train_total

            val_auc = compute_val_auc(model, val_X, val_R)
            improved = ""
            if val_auc > best_auc:
                best_auc = val_auc
                best_state = {k: v.cpu().clone()
                              for k, v in model.state_dict().items()}
                patience_counter = 0
                improved = " *BEST*"
            else:
                patience_counter += 5

            print(f"    Epoch {epoch+1:3d}/{MAX_EPOCHS}  "
                  f"loss={avg_loss:.4f}  "
                  f"train_acc={train_acc:.4f}  "
                  f"val_AUC={val_auc:.4f}{improved}")

            if patience_counter >= PATIENCE:
                print(f"    Early stopping at epoch {epoch+1}")
                break

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(DEVICE)
    print(f"    Best val AUC: {best_auc:.4f}")
    return model, best_auc


# ======================================================================
# 13. Ensemble Prediction (#7)
# ======================================================================

def ensemble_predict(models, user_ratings, track_meta, test_users,
                     feat_mean, feat_std, feature_indices,
                     global_stats, album_to_tracks, artist_to_tracks,
                     mf_models, neighbor_index,
                     out_path, model_name="ensemble"):
    print(f"\n>>> Predicting with {model_name} ({len(models)} models) ...")
    results = []
```

```python

    with torch.no_grad():
        for uid, candidates in test_users:
            ratings = user_ratings.get(uid, {})
            u_mean, u_std, u_n = compute_user_profile(ratings)

            base_feats = []
            for tid in candidates:
                bf = compute_base_features(
                    ratings, track_meta, tid, u_mean, u_std, u_n,
                    global_stats, album_to_tracks, artist_to_tracks,
                    mf_models, uid, neighbor_index, user_ratings)
                base_feats.append(bf)

            cross = compute_cross_features(track_meta, candidates)

            X = np.zeros((1, 6, N_FEATURES), dtype=np.float32)
            for c in range(6):
                X[0, c, :N_BASE_FEATURES] = base_feats[c]
                X[0, c, N_BASE_FEATURES:] = cross[c]

            X = normalize(X, feat_mean, feat_std)
            if feature_indices is not None:
                X = apply_feature_mask(X, feature_indices)

            X_t = torch.tensor(X, dtype=torch.float32, device=DEVICE)

            # Average scores across ensemble
            avg_scores = np.zeros(6)
            for model in models:
                model.eval()
                scores = model(X_t).cpu().numpy()[0]
                avg_scores += scores
            avg_scores /= len(models)

            ranked = sorted(zip(avg_scores, candidates), reverse=True)
            top3 = set(tid for _, tid in ranked[:3])
            for tid in candidates:
                results.append((f"{uid}_{tid}", 1 if tid in top3 else 0))

    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")

    ones = sum(v for _, v in results)
    print(f"    {len(results):,} predictions ({ones:,} rec, "
```

```
          f"{len(results)-ones:,} don't)")
    print(f"    Saved -> {out_path}")
    return results


# ======================================================================
# 14. V2 Baseline
# ======================================================================

def v2_heuristic_val_auc(user_ratings, track_meta, val_groups):
    correct = 0
    total = 0
    for uid, tids, rats in val_groups:
        ratings = user_ratings[uid]
        scores = []
        for tid in tids:
            alb_id, art_id, genre_ids = track_meta.get(tid, (None, None,
                []))
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            scores.append(s)
        for p in range(3):
            for n in range(3, 6):
                if scores[p] > scores[n]:
                    correct += 1
                elif scores[p] == scores[n]:
                    correct += 0.5
                total += 1
    return correct / total if total > 0 else 0.0


# ======================================================================
# 15. Main
# ======================================================================

def main():
    tee = Tee(RESULTS_FILE)
    sys.stdout = tee
    t0 = time.time()

    torch.manual_seed(SEED)
```

```
    random.seed(SEED)
    np.random.seed(SEED)

    print("=" * 60)
    print("  EE627 Midterm v6 - Refined Model")
    print("=" * 60)
    print(f"  Device: {DEVICE}")
    if torch.cuda.is_available():
        print(f"  GPU: {torch.cuda.get_device_name(0)}")
    print()

    # ---- Load data ----
    user_ratings = parse_training(os.path.join(DATA_DIR, "trainItem2.txt")
        )
    track_meta = parse_tracks(os.path.join(DATA_DIR, "trackData2.txt"))
    album_meta = parse_albums(os.path.join(DATA_DIR, "albumData2.txt"))
    test_users = parse_test(os.path.join(DATA_DIR, "testItem2.txt"))

    # ---- Enrich ----
    print("\n" + "=" * 60)
    print("  ENRICHING TRACKS")
    print("=" * 60)
    enriched_meta = enrich_tracks_with_albums(track_meta, album_meta)

    # ---- Auxiliary structures ----
    print("\n" + "=" * 60)
    print("  BUILDING AUXILIARY STRUCTURES")
    print("=" * 60)
    global_stats = build_global_item_stats(user_ratings)
    album_to_tracks, artist_to_tracks = build_track_mappings(enriched_meta
        )

    # ---- (#1, #3) Multiple MF models ----
    print("\n" + "=" * 60)
    print("  TRAINING MF MODELS")
    print("=" * 60)
    mf_models = []
    for cfg in MF_CONFIGS:
        result = train_mf(user_ratings, cfg["factors"], cfg["epochs"],
                          cfg["lr"], MF_BATCH, cfg["label"])
        mf_models.append(result)

    # ---- (#5) Neighbor index from larger MF ----
    print("\n" + "=" * 60)
    print("  BUILDING NEIGHBOR INDEX")
    print("=" * 60)
    # Use the larger MF (mf128) for neighbor similarity
```

```
    mf128_uid_map, _, mf128_user_f, _ = mf_models[1]
    neighbor_index = build_neighbor_index(
        mf128_uid_map, mf128_user_f, user_ratings, N_NEIGHBORS)

    # ---- Split users ----
    print("\n" + "=" * 60)
    print("  BUILDING TRAINING & VALIDATION DATA")
    print("=" * 60)
    rng = random.Random(SEED)
    all_uids = list(user_ratings.keys())
    rng.shuffle(all_uids)
    split = int(len(all_uids) * 0.9)
    train_uids = set(all_uids[:split])
    val_uids = set(all_uids[split:])
    print(f"  Train users: {len(train_uids):,}, Val users: {len(val_uids)
        :,}")

    t1 = time.time()
    train_groups = build_groups(user_ratings, enriched_meta, train_uids,
                                GROUPS_PER_USER, SEED)
    val_groups = build_groups(user_ratings, enriched_meta, val_uids,
                              10, SEED + 1)
    print(f"  Train groups: {len(train_groups):,}, "
          f"Val groups: {len(val_groups):,}")

    print("\n  Featurizing training groups ...")
    train_X, train_R = featurize_groups(
        train_groups, user_ratings, enriched_meta, global_stats,
        album_to_tracks, artist_to_tracks, mf_models, neighbor_index)

    print("  Featurizing validation groups ...")
    val_X, val_R = featurize_groups(
        val_groups, user_ratings, enriched_meta, global_stats,
        album_to_tracks, artist_to_tracks, mf_models, neighbor_index)

    t2 = time.time()
    print(f"  Feature computation: {t2-t1:.1f}s")

    # Normalize
    feat_mean, feat_std = compute_norm_stats(train_X)
    train_X_norm = normalize(train_X, feat_mean, feat_std)
    val_X_norm = normalize(val_X, feat_mean, feat_std)

    # ---- V2 baseline ----
    print("\n" + "=" * 60)
    print("  V2 HEURISTIC BASELINE")
    print("=" * 60)
```

```
    v2_auc = v2_heuristic_val_auc(user_ratings, enriched_meta, val_groups)
    print(f"  v2 heuristic val AUC: {v2_auc:.4f}  (Kaggle: 0.871)")

    # ---- Phase 1: Train initial model for feature importance ----
    print("\n" + "=" * 60)
    print("  PHASE 1: INITIAL MODEL (all {N_FEATURES} features)")
    print("=" * 60)
    model_full, full_auc = train_ranking_model(
        train_X_norm, train_R, val_X_norm, val_R, N_FEATURES)

    # ---- (#2) Feature selection ----
    print("\n" + "=" * 60)
    print("  FEATURE SELECTION")
    print("=" * 60)
    importances = feature_importance(model_full, val_X_norm, val_R)
    selected = select_features(None, importances)
    print(f"\n  Selected {len(selected)}/{N_FEATURES} features: {selected}
        ")

    # Apply mask
    train_X_sel = apply_feature_mask(train_X_norm, selected)
    val_X_sel = apply_feature_mask(val_X_norm, selected)
    n_sel = len(selected)

    # ---- Phase 2: Retrain with selected features ----
    print("\n" + "=" * 60)
    print(f"  PHASE 2: RETRAIN WITH {n_sel} SELECTED FEATURES")
    print("=" * 60)
    model_sel, sel_auc = train_ranking_model(
        train_X_sel, train_R, val_X_sel, val_R, n_sel)

    print(f"\n  Full model AUC:     {full_auc:.4f}")
    print(f"  Selected model AUC: {sel_auc:.4f}")

    # Decide which features to use
    if sel_auc >= full_auc - 0.002:
        use_selected = True
        final_features = selected
        print(f"  Using selected features ({n_sel})")
    else:
        use_selected = False
        final_features = None
        print(f"  Keeping all features ({N_FEATURES})")

    # ---- (#7) Ensemble: train N models with different seeds ----
    print("\n" + "=" * 60)
    print(f"  ENSEMBLE: TRAINING {N_ENSEMBLE} MODELS")
```

```
    print("=" * 60)

    if use_selected:
        ens_train_X = train_X_sel
        ens_val_X = val_X_sel
        ens_n_feat = n_sel
    else:
        ens_train_X = train_X_norm
        ens_val_X = val_X_norm
        ens_n_feat = N_FEATURES

    ensemble_models = []
    ensemble_aucs = []
    for i in range(N_ENSEMBLE):
        print(f"\n  --- Ensemble model {i+1}/{N_ENSEMBLE} (seed offset={i
            *100}) ---")
        model_i, auc_i = train_ranking_model(
            ens_train_X, train_R, ens_val_X, val_R, ens_n_feat,
            seed_offset=i * 100)
        ensemble_models.append(model_i)
        ensemble_aucs.append(auc_i)
        print(f"    Model {i+1} val AUC: {auc_i:.4f}")

    print(f"\n  Ensemble individual AUCs: {ensemble_aucs}")
    print(f"  Mean: {np.mean(ensemble_aucs):.4f}")

    # ---- Generate submissions ----
    print("\n" + "=" * 60)
    print("  GENERATING SUBMISSIONS")
    print("=" * 60)

    v6_results = ensemble_predict(
        ensemble_models, user_ratings, enriched_meta, test_users,
        feat_mean, feat_std, final_features,
        global_stats, album_to_tracks, artist_to_tracks,
        mf_models, neighbor_index,
        os.path.join(OUTPUT_DIR, "submission_v6_ensemble.csv"),
        f"v6 Ensemble ({N_ENSEMBLE} models)")

    # ---- Compute v2 scores per user for hybrid submissions ----
    print("\n" + "=" * 60)
    print("  HYBRID SUBMISSIONS (multiple thresholds)")
    print("=" * 60)

    # Build v6 results lookup by user
    v6_by_user = {}
    idx = 0
```

```
    for uid, candidates in test_users:
        v6_by_user[uid] = {}
        for tid in candidates:
            v6_by_user[uid][tid] = v6_results[idx][1]
            idx += 1

    # Compute v2 scores and predictions per user
    v2_user_data = {}  # uid -> (v2_top3_set, confidence_gap, v2_results_list)
    v2_results = []
    for uid, candidates in test_users:
        ratings = user_ratings.get(uid, {})
        v2_scores = []
        for tid in candidates:
            alb_id, art_id, genre_ids = enriched_meta.get(
                tid, (None, None, []))
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            v2_scores.append(s)

        ranked = sorted(zip(v2_scores, candidates), reverse=True)
        v2_top3 = set(tid for _, tid in ranked[:3])
        sorted_scores = sorted(v2_scores, reverse=True)
        gap = sorted_scores[2] - sorted_scores[3]
        v2_user_data[uid] = (v2_top3, gap)

        for tid in candidates:
            v2_results.append((f"{uid}_{tid}", 1 if tid in v2_top3 else 0))

    # Pure v6 agreement with v2
    agree = sum(1 for (_, a), (_, b) in zip(v6_results, v2_results) if a == b)
    total_preds = len(v6_results)
    agree_pct = 100 * agree / total_preds
    print(f"\n  Pure v6 vs v2 agreement: {agree:,}/{total_preds:,} ({agree_pct:.1f}%)")

    # Generate hybrids at multiple thresholds
    # v5b hybrid with gap>5 scored 0.908, so test around that
    thresholds = [0, 1, 2, 3, 5, 8, 10, 15, 20, 30, 50]
```

```python

    print(f"\n  {'Threshold':>10}  {'v2 users':>10}  {'v6 users':>10}  "
          f"{'v6 pct':>8}  {'Agreement':>10}  File")
    print("  " + "-" * 75)

    for thresh in thresholds:
        hybrid = []
        n_v2 = 0
        n_v6 = 0

        for uid, candidates in test_users:
            v2_top3, gap = v2_user_data[uid]

            if gap > thresh:
                # v2 confident -- keep v2
                n_v2 += 1
                for tid in candidates:
                    hybrid.append(
                        (f"{uid}_{tid}", 1 if tid in v2_top3 else 0))
            else:
                # v2 uncertain -- use v6 model
                n_v6 += 1
                for tid in candidates:
                    hybrid.append(
                        (f"{uid}_{tid}", v6_by_user[uid][tid]))

        # Write submission
        fname = f"submission_v6_hybrid_gap{thresh}.csv"
        fpath = os.path.join(OUTPUT_DIR, fname)
        with open(fpath, "w") as f:
            f.write("TrackID,Predictor\n")
            for key, label in hybrid:
                f.write(f"{key},{label}\n")

        # Agreement with v2
        h_agree = sum(1 for (_, a), (_, b) in zip(hybrid, v2_results)
                      if a == b)
        h_agree_pct = 100 * h_agree / total_preds
        v6_pct = 100 * n_v6 / len(test_users)

        print(f"  {thresh:>10}  {n_v2:>10,}  {n_v6:>10,}  "
              f"{v6_pct:>7.1f}%  {h_agree_pct:>9.1f}%  {fname}")

    # ---- Summary ----
    elapsed = time.time() - t0
    print("\n" + "=" * 60)
    print("  FINAL RESULTS")
```

```
    print("=" * 60)
    print(f"  v2 heuristic val AUC:  {v2_auc:.4f}  (Kaggle: 0.871)")
    print(f"  v5b hybrid (gap>5):    0.908    (Kaggle best so far)")
    print(f"  v6 full model AUC:     {full_auc:.4f}")
    print(f"  v6 selected model AUC: {sel_auc:.4f}")
    print(f"  v6 ensemble AUCs:      {ensemble_aucs}")
    print(f"  Features used:         {ens_n_feat} "
          f"({'selected' if use_selected else 'all'})")
    print(f"  Pure v6 agreement:     {agree_pct:.1f}%")
    print(f"  MF models:             {[c['label'] for c in MF_CONFIGS]}")
    print(f"  Neighbors:             {N_NEIGHBORS}")
    print(f"  Ensemble size:         {N_ENSEMBLE}")
    print()
    print("  Submissions generated:")
    print(f"    submission_v6_ensemble.csv         -- pure v6 model")
    for thresh in thresholds:
        print(f"    submission_v6_hybrid_gap{thresh}.csv"
              f"{' ':>{4-len(str(thresh))}}-- v2 if gap>{thresh}, else v6"
              )
    print()
    print(f"  Best bet: start with gap5 (matched v5b's 0.908 threshold)")
    print(f"  Then try gap3 and gap8 to find the sweet spot")
    print(f"  Total time: {elapsed:.1f}s ({elapsed/60:.1f}m)")
    print("\nDone.")

    tee.close()


if __name__ == "__main__":
    main()
```

Listing 4: EE627_Midterm_v6_refined.py

A.5 Hybrid Threshold Sweep Generator

```python
"""
Generate hybrid submissions at multiple thresholds using v5b's predictions.
v5b hybrid at gap>5 scored 0.908 -- sweep to find the optimal threshold.
"""

import os
import csv
from collections import defaultdict

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "Data", "music-recommender-2026s")

V5B_PURE = os.path.join(SCRIPT_DIR, "submission_v5b_nn.csv")
BASELINE = os.path.join(SCRIPT_DIR, "Part 1 Results", "submission_p1_evidence.csv")
TRAIN_FILE = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE = os.path.join(DATA_DIR, "albumData2.txt")


def parse_training(path):
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    return users


def parse_tracks(path):
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
```

```
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    return tracks


def parse_albums(path):
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
            art = None if len(parts) < 2 or parts[1] == "None" else int(parts[1])
            genres = []
            for g in parts[2:]:
                g = g.strip()
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    return albums


def enrich_tracks(tracks, albums):
    enriched = {}
    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
                new_art = album_art
            track_genre_set = set(genres)
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
        enriched[tid] = (alb, new_art, new_genres)
    return enriched


def parse_test(path):
    test = []
```

```
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    return test


def read_submission(path):
    rows = {}
    with open(path, "r") as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
            rows[row[0]] = int(row[1])
    return rows


def main():
    print("Loading data ...")
    user_ratings = parse_training(TRAIN_FILE)
    track_meta = parse_tracks(TRACK_FILE)
    album_meta = parse_albums(ALBUM_FILE)
    enriched = enrich_tracks(track_meta, album_meta)
    test_users = parse_test(TEST_FILE)

    print("Loading v5b predictions ...")
    v5b_preds = read_submission(V5B_PURE)

    # Compute v2 scores and confidence gaps per user
    print("Computing v2 scores ...")
    thresholds = [0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20]

    # Pre-compute v2 per user
    user_data = []  # (uid, candidates, v2_top3, gap)
    for uid, candidates in test_users:
        ratings = user_ratings.get(uid, {})
        v2_scores = []
```

```
        for tid in candidates:
            alb_id, art_id, genre_ids = enriched.get(tid, (None, None, [])
                )
            s = 0.0
            if alb_id is not None and alb_id in ratings:
                s += ratings[alb_id]
            if art_id is not None and art_id in ratings:
                s += ratings[art_id]
            gs = [ratings[g] for g in genre_ids if g in ratings]
            if gs:
                s += len(gs) * (sum(gs) / len(gs)) / 10.0
            v2_scores.append(s)

        ranked = sorted(zip(v2_scores, candidates), reverse=True)
        v2_top3 = set(tid for _, tid in ranked[:3])
        ss = sorted(v2_scores, reverse=True)
        gap = ss[2] - ss[3]
        user_data.append((uid, candidates, v2_top3, gap))

    # Generate hybrids
    print(f"\n{'Thresh':>6}  {'v2 users':>10}  {'v5b users':>10}  "
          f"{'v5b pct':>8}  {'v2 agree':>10}  File")
    print("-" * 75)

    for thresh in thresholds:
        rows = []
        n_v2 = 0
        n_v5b = 0

        for uid, candidates, v2_top3, gap in user_data:
            if gap > thresh:
                n_v2 += 1
                for tid in candidates:
                    key = f"{uid}_{tid}"
                    rows.append((key, 1 if tid in v2_top3 else 0))
            else:
                n_v5b += 1
                for tid in candidates:
                    key = f"{uid}_{tid}"
                    rows.append((key, v5b_preds[key]))

        fname = f"submission_v5b_hybrid_gap{thresh}.csv"
        fpath = os.path.join(SCRIPT_DIR, fname)
        with open(fpath, "w") as f:
            f.write("TrackID,Predictor\n")
            for key, label in rows:
                f.write(f"{key},{label}\n")
```

```python

        # Agreement with v2
        v2_agree = sum(1 for uid, candidates, v2_top3, _ in user_data
                       for tid in candidates
                       if (v5b_preds.get(f"{uid}_{tid}", 0)
                           == (1 if tid in v2_top3 else 0)))
        # Actually compute for this hybrid
        total = len(rows)
        v5b_pct = 100 * n_v5b / len(user_data)

        print(f"{thresh:>6}  {n_v2:>10,}  {n_v5b:>10,}  "
              f"{v5b_pct:>7.1f}%  {'':>10}  {fname}")

    print(f"\nDone. {len(thresholds)} hybrid submissions generated.")
    print(f"\nv5b hybrid gap5 scored 0.908 on Kaggle.")
    print(f"Try gap3, gap4, gap6, gap7 to bracket the optimum.")


if __name__ == "__main__":
    main()
```

Listing 5: generate_v5b_hybrids.py

A.6 Score Optimization Tool

```python
"""
Score Optimization via Leaderboard Probing
Jude Eschete

Strategy: small batch flips with binary search refinement.
- Flip small batches of users (50 at a time)
- If score goes UP: those users include some wrong predictions --
  keep the batch, then binary search within it to isolate winners
- If score goes DOWN: those users were mostly correct, skip them
- If score SAME: inconclusive, skip and move on

Interactive terminal app. Just type the Kaggle score after uploading.
"""

import os
import sys
import json
import csv
import random
import shutil
from collections import defaultdict

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MIDTERM_DIR = os.path.dirname(SCRIPT_DIR)
BASELINE_CSV = os.path.join(
    MIDTERM_DIR, "Part 1 Results", "submission_p1_evidence.csv")

STATE_FILE = os.path.join(SCRIPT_DIR, "state.json")
BEST_CSV = os.path.join(SCRIPT_DIR, "current_best.csv")
SUBMISSIONS_DIR = os.path.join(SCRIPT_DIR, "submissions")

SEED = 42
BATCH_SIZE = 50  # users per probe


# ====================================================================
# Submission I/O
# ====================================================================

def read_submission(path):
    rows = []
    with open(path, "r") as f:
        reader = csv.reader(f)
        next(reader)
        for row in reader:
```

```
            rows.append((row[0], int(row[1])))
    return rows


def write_submission(rows, path):
    with open(path, "w", newline="") as f:
        f.write("TrackID,Predictor\n")
        for key, label in rows:
            f.write(f"{key},{label}\n")


def group_by_user(rows):
    users = defaultdict(list)
    for key, label in rows:
        uid, tid = key.split("_", 1)
        users[uid].append((key, tid, label))
    return dict(users)


def flatten_users(user_dict, user_order):
    rows = []
    for uid in user_order:
        for key, tid, label in user_dict[uid]:
            rows.append((key, label))
    return rows


def get_user_order(rows):
    seen = set()
    order = []
    for key, _ in rows:
        uid = key.split("_", 1)[0]
        if uid not in seen:
            seen.add(uid)
            order.append(uid)
    return order


# ====================================================================
# Flip logic
# ====================================================================

def flip_users(current_rows, uids_to_flip):
    """Generate new submission with specific users flipped."""
    users = group_by_user(current_rows)
    user_order = get_user_order(current_rows)

```

```
    flip_set = set(uids_to_flip)
    for uid in flip_set:
        if uid in users:
            users[uid] = [(key, tid, 1 - label)
                          for key, tid, label in users[uid]]

    return flatten_users(users, user_order)


# ====================================================================
# State management
# ====================================================================

def load_state():
    if os.path.exists(STATE_FILE):
        with open(STATE_FILE, "r") as f:
            return json.load(f)
    return None


def save_state(state):
    with open(STATE_FILE, "w") as f:
        json.dump(state, f, indent=2)


def next_sub_name(state):
    n = state.get("sub_counter", 0) + 1
    state["sub_counter"] = n
    return f"validation_test_sub_{n:03d}.csv"


# ====================================================================
# Commands
# ====================================================================

def cmd_init():
    if not os.path.exists(BASELINE_CSV):
        print(f"  ERROR: Baseline not found: {BASELINE_CSV}")
        return

    shutil.copy2(BASELINE_CSV, BEST_CSV)
    rows = read_submission(BASELINE_CSV)
    user_order = get_user_order(rows)

    # Shuffle users deterministically for probing order
    rng = random.Random(SEED)
    shuffled = user_order[:]
```

```
    rng.shuffle(shuffled)

    state = {
        "best_score": 0.871,
        "all_uids": user_order,
        "probe_queue": shuffled,       # users to test, in order
        "probe_index": 0,              # where we are in the queue
        "confirmed_good": [],          # flipping these hurt or was
           neutral
        "confirmed_bad": [],           # flipping these helped
        "mode": "scan",                # "scan" or "refine"
        "refine_pool": [],             # users to binary search within
        "refine_half": [],             # current half being tested
        "pending_uids": [],            # users flipped in current probe
        "sub_counter": 0,
        "history": [
            {"action": "init", "score": 0.871}
        ],
    }

    # Generate first probe
    batch = shuffled[:BATCH_SIZE]
    state["pending_uids"] = batch
    state["probe_index"] = BATCH_SIZE

    probe_rows = flip_users(rows, batch)
    name = next_sub_name(state)
    write_submission(probe_rows, os.path.join(SUBMISSIONS_DIR, name))

    save_state(state)

    print(f"  Initialized from baseline (0.871)")
    print(f"  Total users: {len(user_order):,}")
    print(f"  Batch size: {BATCH_SIZE}")
    print(f"")
    print(f"  >> Upload submissions/{name} to Kaggle")
    print(f"     Flipping {len(batch)} users")


def cmd_score(score_str):
    state = load_state()
    if state is None:
        print("  Not initialized. Type 'reset' first.")
        return

    score = float(score_str)
    best = state["best_score"]
```

```
    pending = state["pending_uids"]
    mode = state.get("mode", "scan")

    if score > best:
        # ---- IMPROVEMENT ----
        if mode == "scan":
            print(f"\n  IMPROVED: {best:.3f} -> {score:.3f} (+{score-best:.3f})")
            print(f"  {len(pending)} users contained winners!")

            if len(pending) <= 1:
                # Found an individual winner
                print(f"  Found winning user: {pending[0]}")
                state["confirmed_bad"].extend(pending)
                # Update best
                sub_name = f"validation_test_sub_{state['sub_counter']:03d}.csv"
                shutil.copy2(os.path.join(SUBMISSIONS_DIR, sub_name), BEST_CSV)
                state["best_score"] = score
                # Continue scanning
                _generate_next_scan(state)
            else:
                # Keep the improvement, then refine to find which users helped
                sub_name = f"validation_test_sub_{state['sub_counter']:03d}.csv"
                shutil.copy2(os.path.join(SUBMISSIONS_DIR, sub_name), BEST_CSV)
                state["best_score"] = score

                # Now we want to UN-flip half to see if score drops
                # (binary search: which half contains the winners?)
                state["mode"] = "refine"
                state["refine_pool"] = list(pending)
                half = pending[:len(pending)//2]
                state["refine_half"] = half
                state["pending_uids"] = half

                # Generate probe: UN-flip the first half (revert them)
                best_rows = read_submission(BEST_CSV)
                probe_rows = flip_users(best_rows, half)
                name = next_sub_name(state)
                write_submission(probe_rows,
                                 os.path.join(SUBMISSIONS_DIR, name))

                print(f"  Entering refinement mode to isolate winners.")
```

```
                print(f"\n  >> Upload submissions/{name}")
                print(f"     Testing: un-flipping {len(half)} of {len(pending)} users")

        elif mode == "refine":
            # Un-flipping this half improved score further --
            # means this half was hurting, the OTHER half has the winners
            print(f"\n  REFINE IMPROVED: {best:.3f} -> {score:.3f}")
            sub_name = f"validation_test_sub_{state['sub_counter']:03d}.csv"
            shutil.copy2(os.path.join(SUBMISSIONS_DIR, sub_name), BEST_CSV)
            state["best_score"] = score

            pool = state["refine_pool"]
            tested_half = state["refine_half"]
            other_half = [u for u in pool if u not in set(tested_half)]

            # The tested half was bad (un-flipping helped), mark as good
            state["confirmed_good"].extend(tested_half)

            if len(other_half) <= 2:
                # Small enough, mark remaining as bad and move on
                state["confirmed_bad"].extend(other_half)
                state["mode"] = "scan"
                _generate_next_scan(state)
            else:
                # Continue refining the other half
                state["refine_pool"] = other_half
                half = other_half[:len(other_half)//2]
                state["refine_half"] = half
                state["pending_uids"] = half

                best_rows = read_submission(BEST_CSV)
                probe_rows = flip_users(best_rows, half)
                name = next_sub_name(state)
                write_submission(probe_rows,
                                 os.path.join(SUBMISSIONS_DIR, name))

                print(f"  Narrowing: {len(other_half)} users remain")
                print(f"\n  >> Upload submissions/{name}")
                print(f"     Un-flipping {len(half)} users")

        state["history"].append({
            "action": "improved", "score": score,
            "delta": score - best, "users": len(pending), "mode": mode
        })
```

```python

    elif score < best:
        # ---- REGRESSION ----
        if mode == "scan":
            print(f"\n  REGRESSED: {best:.3f} -> {score:.3f} ({score-best:.3f})")
            print(f"  Those {len(pending)} users were correct, skipping.")
            state["confirmed_good"].extend(pending)
            _generate_next_scan(state)

        elif mode == "refine":
            # Un-flipping this half made it worse --
            # means this half contained winners, keep them flipped
            print(f"\n  REFINE REGRESSED: {best:.3f} -> {score:.3f}")
            print(f"  This half contains winners -- keeping them.")

            pool = state["refine_pool"]
            tested_half = state["refine_half"]
            other_half = [u for u in pool if u not in set(tested_half)]

            # The other half might be hurting -- mark as good (revert-worthy)
            state["confirmed_good"].extend(other_half)

            if len(tested_half) <= 2:
                state["confirmed_bad"].extend(tested_half)
                state["mode"] = "scan"
                _generate_next_scan(state)
            else:
                # Narrow within the winning half
                state["refine_pool"] = tested_half
                half = tested_half[:len(tested_half)//2]
                state["refine_half"] = half
                state["pending_uids"] = half

                best_rows = read_submission(BEST_CSV)
                probe_rows = flip_users(best_rows, half)
                name = next_sub_name(state)
                write_submission(probe_rows,
                                 os.path.join(SUBMISSIONS_DIR, name))

                print(f"  Narrowing: {len(tested_half)} users remain")
                print(f"\n  >> Upload submissions/{name}")
                print(f"     Un-flipping {len(half)} users")

        state["history"].append({
            "action": "regressed", "score": score,
```

```
            "delta": score - best, "users": len(pending), "mode": mode
        })

    else:
        # ---- SAME SCORE ----
        print(f"\n  NEUTRAL: {score:.3f} (same as best)")
        if mode == "scan":
            print(f"  Flipping {len(pending)} users made no difference, skipping.")
            state["confirmed_good"].extend(pending)
            _generate_next_scan(state)
        elif mode == "refine":
            print(f"  Inconclusive refinement. Moving on.")
            state["confirmed_good"].extend(state["refine_pool"])
            state["mode"] = "scan"
            _generate_next_scan(state)

        state["history"].append({
            "action": "neutral", "score": score,
            "users": len(pending), "mode": mode
        })

    save_state(state)


def _generate_next_scan(state):
    """Generate the next scan probe from the queue."""
    queue = state["probe_queue"]
    idx = state["probe_index"]
    good_set = set(state["confirmed_good"])
    bad_set = set(state["confirmed_bad"])

    # Find next untested batch
    batch = []
    while idx < len(queue) and len(batch) < BATCH_SIZE:
        uid = queue[idx]
        idx += 1
        if uid not in good_set and uid not in bad_set:
            batch.append(uid)

    state["probe_index"] = idx
    state["pending_uids"] = batch
    state["mode"] = "scan"

    if not batch:
        n_good = len(state["confirmed_good"])
        n_bad = len(state["confirmed_bad"])
```

```
        print(f"\n  ALL USERS TESTED!")
        print(f"  Confirmed correct: {n_good:,}")
        print(f"  Confirmed wrong:   {n_bad:,}")
        print(f"  Best score: {state['best_score']:.4f}")
        print(f"  Final submission: current_best.csv")
        return

    best_rows = read_submission(BEST_CSV)
    probe_rows = flip_users(best_rows, batch)
    name = next_sub_name(state)
    write_submission(probe_rows, os.path.join(SUBMISSIONS_DIR, name))

    n_good = len(state["confirmed_good"])
    n_bad = len(state["confirmed_bad"])
    n_total = len(state["all_uids"])
    progress = 100 * (n_good + n_bad) / n_total

    print(f"\n  Best: {state['best_score']:.4f}  "
          f"Progress: {n_good+n_bad:,}/{n_total:,} ({progress:.0f}%)")
    print(f"  Correct: {n_good:,}  Wrong: {n_bad:,}")
    print(f"\n  >> Upload submissions/{name}")
    print(f"     Flipping {len(batch)} users")


def cmd_status():
    state = load_state()
    if state is None:
        print("  Not initialized. Type 'reset'.")
        return

    n_good = len(state.get("confirmed_good", []))
    n_bad = len(state.get("confirmed_bad", []))
    n_total = len(state["all_uids"])

    print()
    print("=" * 50)
    print("  STATUS")
    print("=" * 50)
    print(f"  Best score:    {state['best_score']:.4f}")
    print(f"  Mode:          {state.get('mode', 'scan')}")
    print(f"  Submissions:   {state.get('sub_counter', 0)}")
    print(f"  Correct users: {n_good:,} ({100*n_good/n_total:.1f}%)")
    print(f"  Wrong users:   {n_bad:,} ({100*n_bad/n_total:.1f}%)")
    print(f"  Untested:      {n_total-n_good-n_bad:,}")
    print(f"  Pending flip:  {len(state.get('pending_uids', [])):,}")
    print()
    print("  Recent history:")
```

```
    for h in state.get("history", [])[-10:]:
        action = h["action"]
        score = h.get("score", "")
        delta = h.get("delta", "")
        d = f" ({delta:+.3f})" if delta else ""
        n = h.get("users", "")
        mode = h.get("mode", "")
        print(f"    {action:12s} {score}{d}  "
              f"[{n} users] {mode}")


# ====================================================================
# Interactive Loop
# ====================================================================

def main():
    os.makedirs(SUBMISSIONS_DIR, exist_ok=True)

    print("=" * 50)
    print("  SCORE OPTIMIZATION")
    print("=" * 50)

    state = load_state()
    if state is None:
        print("  No state found. Initializing from baseline...")
        print()
        cmd_init()

    print()
    print("  Commands: <score>, status, reset, quit")
    print()

    while True:
        try:
            user_input = input(">> ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nBye.")
            break

        if not user_input:
            continue

        if user_input.lower() in ("quit", "exit", "q"):
            print("Bye.")
            break
        elif user_input.lower() == "status":
            cmd_status()
```

```
        elif user_input.lower() == "reset":
            confirm = input("   Reset all progress? (y/n): ").strip().
               lower()
            if confirm == "y":
                # Clean old submissions
                for f in os.listdir(SUBMISSIONS_DIR):
                    os.remove(os.path.join(SUBMISSIONS_DIR, f))
                cmd_init()
            else:
                print("   Cancelled.")
        else:
            try:
                score = float(user_input)
                if 0 <= score <= 1:
                    cmd_score(user_input)
                else:
                    print("  Score should be between 0 and 1")
            except ValueError:
                print(f"  Unknown: {user_input}")
                print("  Enter a score, 'status', 'reset', or 'quit'")


if __name__ == "__main__":
    main()
```

Listing 6: score_optimization/optimize.py